



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий
Кафедра математического обеспечения информационных систем

РАБОТА ДОПУЩЕНА К ЗАЩИТЕ

Заведующий
кафедрой _____ С.И. Довгучиц

«02» _____ июня _____ 2025 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

по направлению подготовки магистров 09.04.01 Архитектура вычислительной техники
и информационных систем

На тему: Разработка системы автономной верификации устройства встроенного
самотестирования памяти

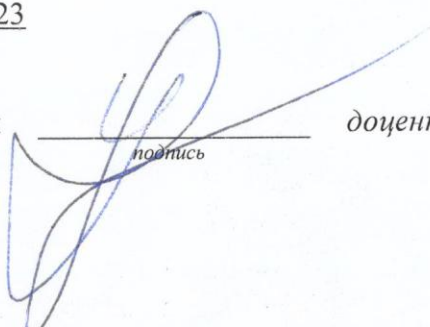
Обучающийся


подпись

Гордеев Михаил Ильич
Фамилия, имя, отчество

шифр 23И1510
группа ИВМО-04-23

Руководитель работы


подпись

доцент

Уманский С.А.

Москва 2025 г.



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий

Кафедра математического обеспечения информационных систем (МОИС)

СОГЛАСОВАНО

Заведующий
кафедрой

подпись

Довгучин Сергей Иванович

«10»

марта

2025 г.

УТВЕРЖДАЮ

Директор
института

подпись

Зуев Андрей Сергеевич

«10»

марта

2025 г.

ЗАДАНИЕ

на выполнение выпускной квалификационной работы магистра
(магистерской диссертации)

Обучающийся

Гордеев Михаил Ильич

Фамилия Имя Отчество

Шифр

23И1510

Направление
подготовки

09.04.01

Информатика и вычислительная техника

индекс направления

наименование направления

Группа

ИВМО-04-23

1. Тема выпускной квалификационной работы: Разработка системы автономной верификации устройства встроенного самотестирования памяти

2. Цель и задачи выпускной квалификационной работы

Цель работы: обеспечить верификацию устройства до его выпуска в физическом виде.

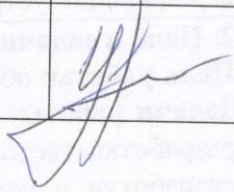
Задачи работы: изучение существующих методов и подходов в автономной верификации; разработка тестовой системы для автономной верификации; обоснование выбора средств разработки и реализации тестовой системы; реализация основных компонентов тестовой системы, включая библиотеки последовательностей, алгоритмы генерации воздействий и анализатора данных.

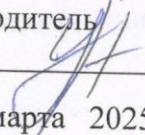
3. Этапы выпускной квалификационной работы

№ этапа	Содержание этапа выпускной квалификационной работы	Результат выполнения этапа ВКР	Срок выполнения
1	Объект, предмет, проблема и цель исследования		
1.1	Анализ существующих методов и подходов в автономной верификации устройств		
1.2	Анализ предмета, объекта и проблемы исследования		
1.2.1	Определение цели и задач исследования		
1.2.2	Предпроектное обследование объекта исследования		
1.2.2	Анализ информационных процессов в области верификации устройств		22.03.2025
1.2.3	Сравнительный анализ аналогов средств автоматизации в области верификации устройств		
1.2.4	Постановка задачи на разработку		
2	Организационное обеспечение проекта		
2.1	Анализ вариантов решения проблемы исследования		
2.2	Инфраструктура информационных технологий проекта		
2.2.1	Системное и прикладное программное обеспечение		
2.2.2	Инструментальное программное обеспечение		
2.2.3	Техническое обеспечение		
2.3	Планирование реализации проекта		
2.3.1	Резюме проекта		
2.3.2	Иерархическая структура работ проекта		
2.3.3	Оценка стоимости проекта		
2.3.4	Расписание проекта		
2.3.5	Формирование календарно-ресурсного плана проекта		19.04.2025
3	Результаты проектирования информационной системы		
3.1	Сценарии работы пользователей		
3.1.1	Варианты использования системы		
3.1.2	Формы документов		
3.2	Информационное обеспечение		
3.2.1	Выбор и обоснование архитектуры информационной базы		
3.2.2	Модели информационной базы		16.05.2025
4	Введение, заключение, список источников, приложения		17.05.2025
5	Презентационный материал, аннотация		26.05.2025
6	Нормоконтроль		27.05.2025

4. Перечень разрабатываемых документов и графических материалов: электронная версия магистерской диссертации, презентационный материал с основными результатами выпускной квалификационной работы магистра, написанный код на языке описания аппаратуры SystemVerilog, результат работы тестовой системы, графический материал по исследованию предметной области

5. Руководитель выпускной квалификационной работы

Функциональные обязанности	Должность в Университете	Фамилия, имя, отчество	Подпись
Руководитель ВКР	доцент	Уманский Станислав Александрович	

Задание выдал
Руководитель
ВКР: 

«10» марта 2025 г.

подпись

Задание принял к исполнению
Обучающийся: 

«10» марта 2025 г.

подпись

АННОТАЦИЯ

Выпускная квалификационная работа посвящена теме: «Разработка системы автономной верификации устройства встроенного самотестирования памяти». Применение: разработка микропроцессоров.

Выпускная квалификационная работа состоит из введения, трех глав, заключения, списка использованных источников. Во введении раскрыта актуальность темы, поставленные цели и задачи. В первой главе анализ предметной области, описаны функциональные требования к программной реализации. Во второй главе показаны этапы проектирования системы при помощи UML диаграмм, а также представлены организация и планирование работ, расчёт стоимости проведения работ. В третьей главе отображены программная реализация системы, реализация модулей, графический анализ данных. В заключении сделаны выводы о совершенной работе и сформулированы достоинства и недостатки в ходе исследования.

Работа состоит из 79 страниц, включает 14 рисунков, 10 таблиц, 7 листингов программ, 14 страниц приложения.

Ключевые слова: SystemVerilog, верификация, UVM, BIST, память, C++.

The graduation thesis is devoted to the topic: “Development of an Autonomous Verification System for a Memory Built-In Self-Test Device.” Application: Microprocessor development. The graduation thesis consists of an introduction, three chapters, a conclusion, and a list of references. In the introduction, the relevance of the topic is discussed, along with the set goals and objectives. The first chapter provides an analysis of the subject area and describes the functional requirements for the software implementation. The second chapter outlines the stages of system design using UML diagrams, as well as the organization and planning of work, including cost estimation. The third chapter presents the software implementation of the system, module development, and graphical data

analysis. In the conclusion, the results of the work are summarized, and the strengths and weaknesses of the research are highlighted.

The thesis comprises 79 pages, including 14 figures, 10 tables, and 7 code listings 14 pages of supplement.

Keywords: SystemVerilog, verification, UVM, BIST, memory, C++.

СПИСОК ИСПОЛЬЗУЕМЫХ СОКРАЩЕНИЙ

BIST – Built-In Self-Test – система встроенного самотестирования памяти

ПЛИС – Программируемая Логическая Интегральная Схема

JTAG — Joint Test Action Group – стандарт отладки и тестирования

RTL – Register Transfer Level – уровень регистровых передач

HDL – Hardware Description Level – язык описания аппаратуры

UVM – Unified Verification Methodology – единая методология верификации

DPI – Direct Programming Interface – прямой программный интерфейс

WBS – Work Breakdown Structure — иерархическая структура работ

DUT – Design Under Test – Тестируемое устройство

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....	8
1 ОБЪЕКТ, ПРЕДМЕТ, ПРОБЛЕМА И ЦЕЛЬ ИССЛЕДОВАНИЯ	10
1.1 Анализ существующих методов и подходов в автономной верификации устройств.....	10
1.2 Анализ предмета, объекта и проблемы исследования	12
1.2.1 Определение цели и задач исследования	12
1.2.2 Предпроектное обследование объекта исследования	12
1.2.3 Анализ информационных процессов в области верификации устройств	17
1.2.4 Сравнительный анализ аналогов средств автоматизации в области верификации устройств	19
1.2.5 Постановка задачи на разработку.....	22
2 ОРГАНИЗАЦИОННОЕ ОБЕСПЕЧЕНИЕ ПРОЕКТА	23
2.1 Анализ вариантов решения проблемы исследования	23
2.2 Инфраструктура информационных технологий проекта.....	26
2.2.1 Системное и прикладное программное обеспечение	26
2.2.2 Инструментальное программное обеспечение	30
2.2.3 Техническое обеспечение.....	33
2.3 Планирование реализации проекта	36
2.3.1 Резюме проекта	36
2.3.2 Иерархическая структура работ проекта	38
2.3.3 Оценка стоимости проекта	40
2.3.4 Расписание проекта.....	43
2.3.5 Формирование календарно-ресурсного плана проекта.....	45
3 РЕЗУЛЬТАТЫ ПРОЕКТИРОВАНИЯ ИНФОРМАЦИОННОЙ СИСТЕМЫ.....	49

3.1 Сценарии работы пользователей	47
3.1.1 Варианты использования системы	47
3.1.2 Формы документов	50
3.2 Информационное обеспечение	57
3.2.1 Выбор и обоснование архитектуры.....	57
3.2.2 Модели информационной базы	64
3.3 Системы классификации и кодирования	67
3.4 Обеспечение информационной безопасности.....	69
ЗАКЛЮЧЕНИЕ	72
CONCLUSION	75
ИСПОЛЬЗОВАННЫЕ ИСТОЧНИКИ	77
ПРИЛОЖЕНИЕ	79

ВВЕДЕНИЕ

Современные электронные устройства, включая микропроцессоры, системы связи и встраиваемые решения, требуют высокой надежности и отказоустойчивости. Одним из ключевых компонентов таких систем является память, отказ которой может привести к критическим последствиям. Для обеспечения надежной работы памяти применяются методы встроенного самотестирования, позволяющие автоматически обнаруживать дефекты на этапах производства и эксплуатации. Однако эффективность BIST-механизмов напрямую зависит от корректности их работы, что обуславливает необходимость верификации таких систем.

Актуальность данной работы обусловлена возрастающей сложностью интегральных схем и увеличением плотности размещения элементов памяти, что повышает вероятность возникновения дефектов. Традиционные методы тестирования, основанные на внешнем оборудовании, не всегда обеспечивают достаточную скорость и точность диагностики. В связи с этим разработка автономной системы верификации BIST-устройств становится важной задачей, позволяющей повысить надежность и снизить затраты на тестирование.

Целью дипломного проекта является разработка системы автономной верификации устройства встроенного самотестирования памяти, обеспечивающей автоматическую проверку его функциональности и диагностику возможных ошибок. Для достижения поставленной цели необходимо решить следующие задачи:

- анализ предметной области;
- формализация требований к системе автономной верификации;
- разработка архитектуры системы автономной верификации.

Практическая значимость работы заключается в создании системы, способной повысить надежность диагностики памяти за счет верификации

BIST. Результаты проекта будут использованы при проектировании отказоустойчивых вычислительных систем, таких как процессоры семейства «Эльбрус».

Modern electronic devices, including microprocessors, communication systems, and embedded solutions, require high reliability and fault tolerance. One of the key components of such systems is memory, the failure of which can lead to critical consequences. To ensure reliable memory operation, Built-In Self-Test (BIST) methods are used, allowing for automatic defect detection during both production and operation. However, the effectiveness of BIST mechanisms directly depends on their correct operation, which necessitates the verification of such systems.

The relevance of this work is driven by the increasing complexity of integrated circuits and the growing density of memory elements, which raises the likelihood of defects. Traditional testing methods based on external equipment do not always provide sufficient speed and diagnostic accuracy. Therefore, the development of an autonomous BIST device verification system becomes an important task, improving reliability and reducing testing costs.

The objective of this graduation project is to develop an autonomous verification system for a memory BIST device, ensuring automated functionality testing and diagnosis of potential errors. To achieve this goal, the following tasks must be completed:

- Analysis of the subject area;
- Formalization of requirements for the autonomous verification system;
- Development of the autonomous verification system architecture.

The practical significance of this work lies in the creation of a system capable of improving memory diagnostic reliability through BIST verification. The project results will be applied in the design of fault-tolerant computing systems, such as the Elbrus family of processors.

3 ОБЪЕКТ, ПРЕДМЕТ, ПРОБЛЕМА И ЦЕЛЬ ИССЛЕДОВАНИЯ

1.1 Анализ существующих методов и подходов в автономной верификации устройств

Верификация устройств — это процесс подтверждения того, что устройство функционирует в соответствии со спецификацией и проектными требованиями. В контексте цифровой электроники под верификацией подразумевается выявление ошибок в логике работы программного или аппаратного обеспечения до этапа массового производства [1].

Автономная верификация, в свою очередь, это процесс верификации устройства без сторонних модулей, проверка его на соответствие технической документации и стандартам.

Существует несколько методов верификации [2]:

- симуляционное тестирование – в данном случае используется модель устройства, а также тестовые сценарии, которые прогоняются в симуляторе. Из недостатков можно отметить неполноту покрытия, которая сильно зависит от набора тестов;
- формальная верификация – в случае формальной верификации, процесс проверки устройства базируется на математических методах и логических доказательствах. Недостатки – хоть и обеспечивает высокую надежность для несложных систем, но полностью теряет свои преимущества, когда используется для сложных систем со сложными автоматами состояний;
- аппаратная верификация и эмуляция – это проверка на уровне реального или эмулируемого «железа», например с использованием ПЛИС. Недостатки – отсутствие возможности точно определить место возникновения ошибки, только факт ее появления;

- свойства и спецификации – в данном случае в системе задаются некоторые формальные свойства, которые затем проверяются в процессе моделирования. Главным недостатком данного способа является его несамостоятельность, т.к. в основном этот метод является частью конкретного языка. Обычно он используется в купе с прочими методами верификации.

Для реализации процессов верификации разработано множество программных и аппаратных решений, каждое из которых охватывает определённые аспекты проверки цифровых устройств. В таблице 1.1 приведены наиболее распространённые решения, используемые в промышленности или научной среде.

Таблица 1.1 – Инструменты верификации

Инструмент	Тип верификации	Особенности и возможности
ModelSim	Симуляция	Один из самых популярных RTL-симуляторов; поддержка VHDL и Verilog[3]
Synopsys VCS	Симуляция + анализ покрытия	Высокая производительность, интеграция с UVM и OVM, поддержка SVA, покрытие кода [4]
Cadence JasperGold	Формальная верификация	Автоматическая проверка свойств, работа с высокоуровневыми спецификациями [5]
Questa Advanced Simulator	Симуляция + формальная проверка	Поддержка UVM, функциональное покрытие [6]
Verilator	RTL симуляция	Открытое решение с высокой скоростью симуляции, подходит для C++ интеграции [7]
Cocotb	Python-интерфейс к симуляторам	Позволяет писать тестбенчи на Python, интегрируется с ModelSim, Icarus, VCS и др.

Варианты применения указанных инструментов могут зависеть от требований проекта, бюджета и целей разработки.

1.2 Анализ предмета, объекта и проблемы исследования

1.2.1 Определение цели и задач исследования

Целью настоящего исследования является разработка системы автономной верификации BIST – системы встроенного самотестирования памяти, способной выполнять проверку корректности функционирования устройства. Планируемая система должна учитывать архитектурные особенности устройства BIST, обеспечивать адаптивную генерацию тестов, автоматический анализ результатов, формирование отчётности.

Для достижения поставленной цели в ходе работы необходимо выполнить следующие задачи:

- анализ предметной области;
- формализация требований к системе автономной верификации;
- разработка архитектуры системы автономной верификации.

1.2.2 Предпроектное обследование объекта исследования

Объектом исследования является устройство BIST – специализированное семейство модулей, выполняющее функции [8]:

- проверки памяти на наличие в ней ошибок логического характера;
- удаление неиспользуемых строк, столбцов, ячеек памяти по результатам проверки;
- замена столбцов памяти, если такая возможность поддерживается банком памяти;
- запись данных об ошибках в отдельную память, а также анализ этих данных при проверке.

Устройство представляет собой микропроцессорную систему, схематично изображенную на рисунке 1.1.

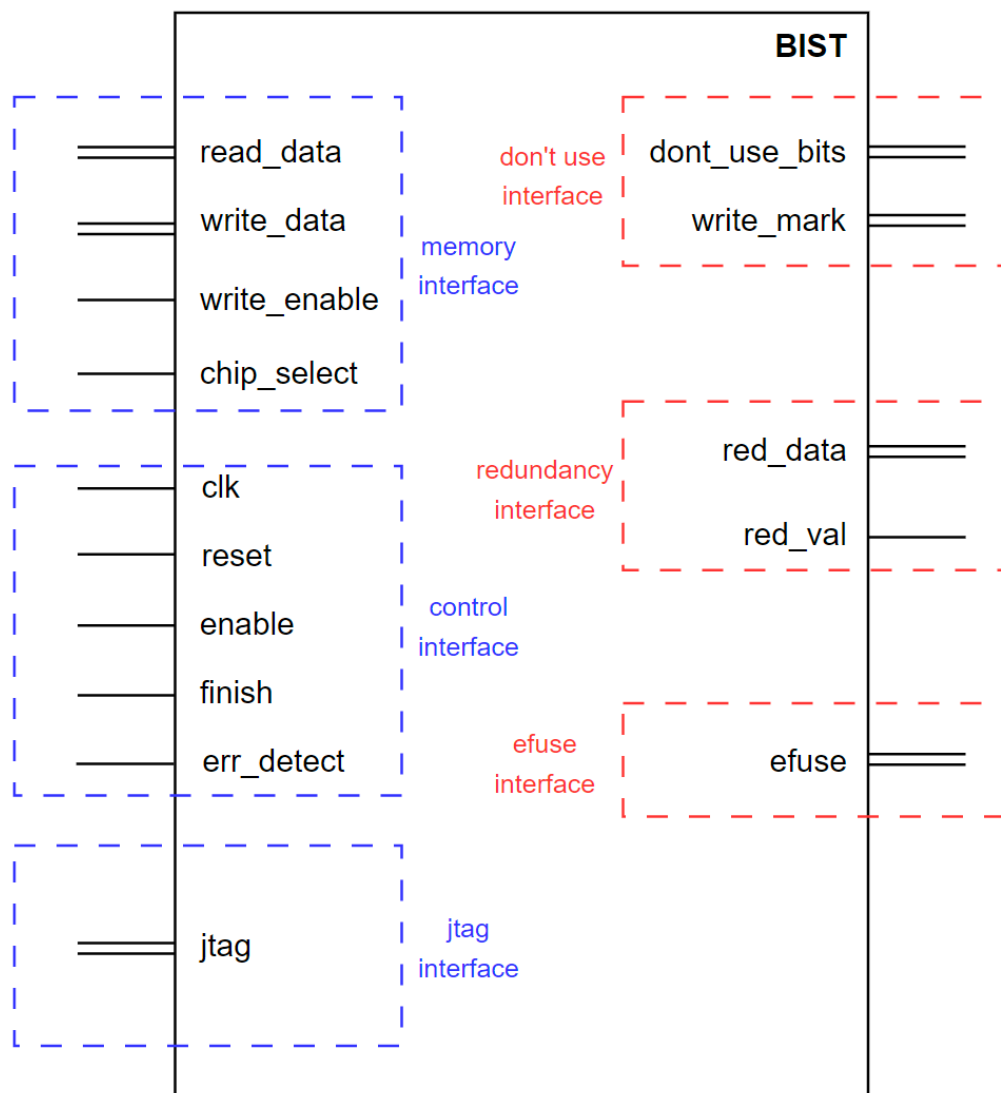


Рисунок 1.1 – Схема BIST

Устройство состоит из следующих частей.

1. Постоянные интерфейсы (выделены синим цветом) – интерфейсы, которые присутствуют на устройстве вне зависимости от его назначения:

- а. memory interface – интерфейс связи с проверяемым банком памяти;
 - б. control interface – интерфейс для работы с самим BIST: запуск, сброс, сигнал об обнаружении ошибки;
 - в. JTAG interface – интерфейс JTAG, служащий для отладки модуля на кристалле.
2. Специфические интерфейсы [9] (выделены красным цветом) – интерфейсы, которые присутствуют только в некоторых модулях, в которых они востребованы:
- а. don't use interface – интерфейс для пометок неиспользуемых элементах памяти;
 - б. redundancy interface – интерфейс для замены столбцов памяти;
 - в. 14ffuse interface – интерфейс связи с памятью для регистрации и анализа ошибок.
3. Основной вычислительный блок (собственно, сам BIST), выполняющий тестирование памяти, используя определенные алгоритмы проверки.

Характерной особенностью устройства является высокая плотность логики и специфичная архитектура, требующая точного соответствия проектной спецификации.

Верификация устройств BIST осложняется рядом факторов:

- наличие множества режимов работы, в зависимости от типа и предназначения устройства;
- большой набор ошибок памяти, каждая из которых предполагает разный подход к симуляции ошибок;
- взаимодействие с внешними устройствами через нестандартные интерфейсы, в частности замена столбцов памяти.

Поэтому верификация должна учитывать:

- обработку сигнальных и логических состояний во времени;

- проверку корректного перехода между состояниями;
- контроль внешних воздействий.

Для формирования требований к будущей системе верификации были проанализированы следующие материалы.

1. Техническая документация на BIST – спецификация, протоколы, форматы данных, а так же отдельно были проанализированы все возможные логические ошибки памяти [10-14] :
 - а. SAF – Stuck At Fault, ячейка постоянно находится в состоянии логического нуля или логической единицы.
 - б. SOF – Stuck Open Fault, ячейка не подсоединена (нет доступа), при записи значение пропадает, при чтении результат зависит от устройства усилителя считывания.
 - в. TF – Transition Fault, ячейка, перейдя в состояние с определенным значением, не может совершить обратный переход.
 - г. WDF – Write Destructive Fault, запись в ячейку уже содержащегося в ней значения может изменить её содержимое;
 - д. RDF – Deceptive Read Destructive Fault, чтение из ячейки может изменить её содержимое, хотя при этом значение, полученное после первого чтения, может быть верным.
 - е. DRF – Data retention Fault, ячейка переходит в стабильное состояние при отсутствии обращений к ней в течение некоторого времени.
 - ж. PF – Parametric Fault, неудовлетворительные технические характеристики ячейки (такие, как потребление мощности, ток утечки, уровни чувствительности к логическим сигналам, время установления значения и т.п.).
 - з. CFst – State Coupling Fault, ячейка переходит в определенное состояние при некотором значении другой ячейки;

- и. CFds – Disturb Coupling Fault, чтение или запись определенного значения по адресу одной ячейки приводит к переходу другой ячейки в определенное состояние.
 - к. Cfid – Idempotent Coupling Fault, изменение содержимого одной ячейки приводит к перевороту значения, содержащегося в другой ячейке.
 - л. Cfin – Inversion Coupling Fault, изменение содержимого одной ячейки вызывает переход другой ячейки в определенное состояние.
 - м. CFtr – Transition Coupling Fault, ячейка, перейдя в состояние с определенным значением, не может совершить обратный переход при определенном значении другой ячейки.
 - н. CFwd – Write Destructive Coupling Fault, запись в ячейку уже содержащегося в ней значения может изменить ее содержимое при определенном значении другой ячейки.
 - о. CFdr – Deceptive Read Destructive Coupling Fault, чтение из ячейки может изменить её содержимое при определенном значении другой ячейки, хотя при этом значение, полученное после первого чтения, может быть верным.
2. Схемотехнические и логические диаграммы, содержащие в себе правильные подключения, варианты использования, временные диаграммы и маршевые последовательности;
 3. Отчеты об ошибках и сбоях на разных этапах разработки.

На основе этого выявлены ключевые сценарии, подлежащие верификации, и ограничения связанные с архитектурой устройства.

После всего были определены следующие требования к разрабатываемой системе:

- поддержка автоматической генерации тестов, для проверки каждой ячейки и воспроизведении в ней каждой возможной логической ошибки;

- минимизация зависимости от разработчика – система должна работать в фоновом режиме, генерируя ошибки самостоятельно, не прибегая к ручному запуску;
- совместимость с эмуляторами и симуляторами, используемыми при проектировании;
- формирование отчетов, включающих в себя информацию о покрытии, ошибках, нарушении временных условий и пр.
- гибкая настройка сценариев тестирования.

1.2.3 Анализ информационных процессов в области верификации устройств

Проектирование аппаратных средств традиционно делится на этапы, приведенные на рисунке 1.2 [15]:

- постановка ТЗ;
- архитектурное проектирование;
- RTL-описание;
- эмуляционная верификация;
- эмуляция на ПЛИС;

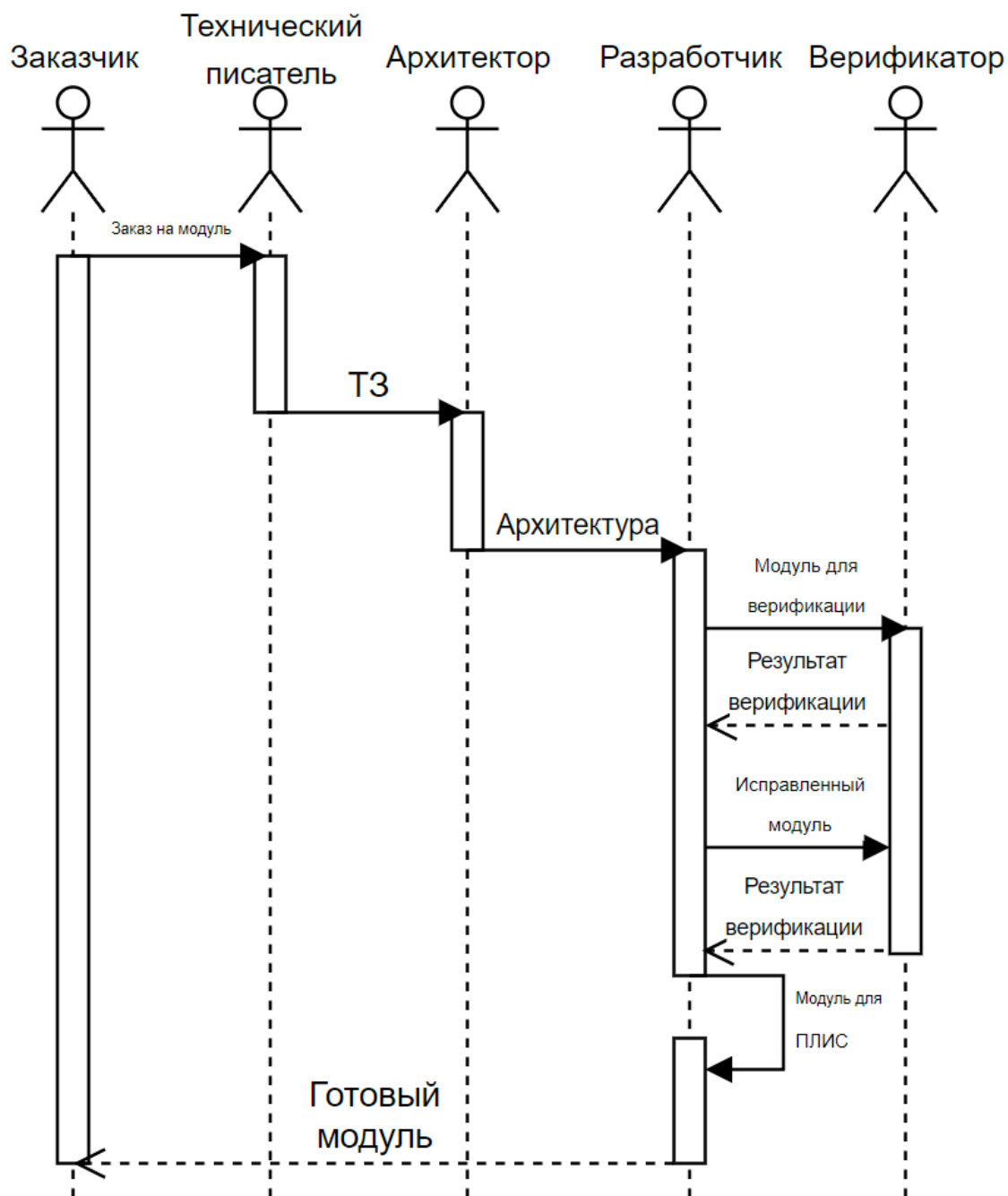


Рисунок 1.2 – Жизненный цикл проектирования модуля

Среди основных информационных потоков и процессов можно выделить следующее[16-17].

1) Входные потоки:

- а. Спецификация устройства.
- б. Описание архитектуры и интерфейсов.
- в. Rtl-код.

г. Тест-кейсы и генераторы тестов.

2) Внутренние процессы и преобразования:

а. Сценарии верификации → тестбенчи → симуляции → лог-файлы → отчёты покрытия.

б. Assertions → runtime-логика → диагностика нарушений.

в. Ограничения → random test generation → входные стимулы.

3) Выходные потоки:

а. Лог-файлы симуляции.

б. Отчёты покрытия.

в. Журналы ошибок и предупреждений.

г. Отчёты регрессионного тестирования.

1.2.4 Сравнительный анализ аналогов средств автоматизации в области верификации устройств

1.2.4.1 Сравнительный анализ языков описания аппаратуры

В настоящее время существует широкий спектр языков описания аппаратуры (HDL) и средств автоматизации проектирования, каждый из которых ориентирован на определённый уровень абстракции и тип задач. Выбор подходящего инструмента верификации напрямую зависит от этапа проектирования, глубины модели, необходимой точности симуляции и специфики устройства.

В рамках анализа были рассмотрены следующие языки и среды (рисунок 1.3):

- Verilog и VHDL — традиционные языки для RTL-описания и логического моделирования. Широко применяются на этапе функциональной симуляции, логического синтеза и верификации на низком уровне абстракции;

- SystemVerilog — расширение Verilog, включающее средства для описания тестбенчей, assertions (SVA), генерации тестов (randomization), что делает его удобным для функциональной верификации и поведения модели;
- SystemC — библиотека C++ для моделирования на системном уровне, чаще используется при архитектурно-алгоритмическом проектировании и поведении на высоком уровне;
- C/C++ — применяются преимущественно на этапах архитектурного и программного моделирования, когда важны алгоритмы и временные характеристики, но не детали реализации на логическом уровне.

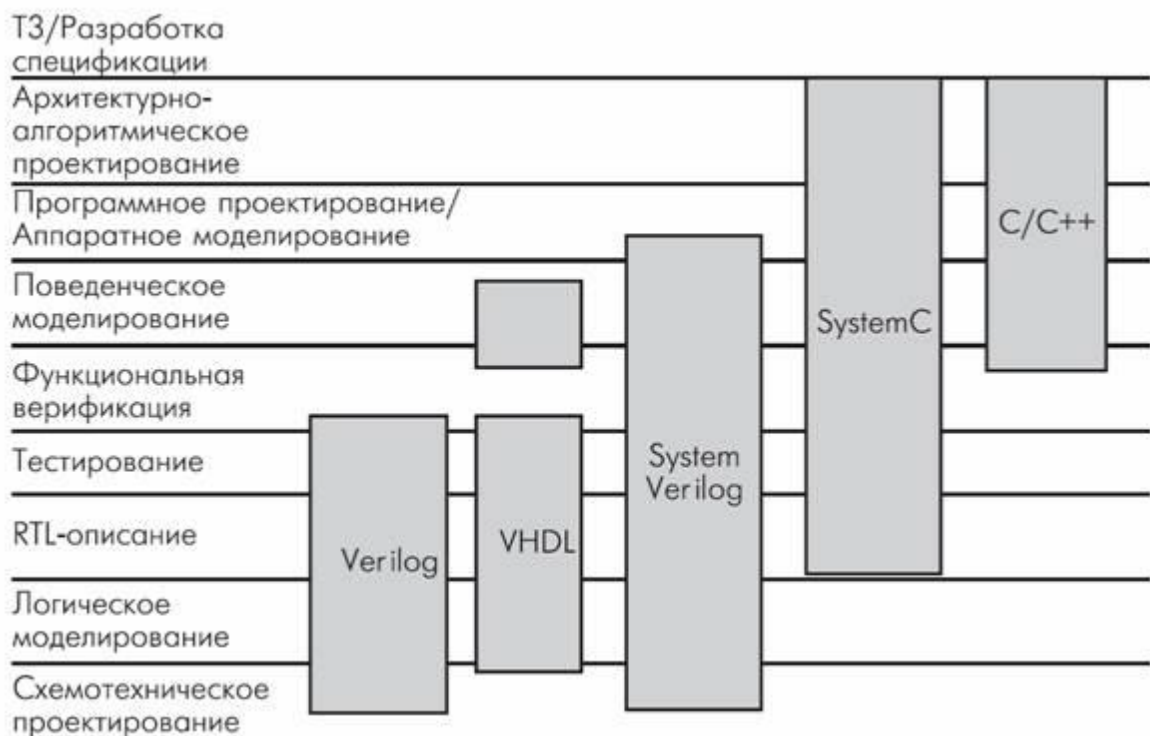


Рисунок 1.3 – Сравнение различных языков описания аппаратуры относительно уровня абстракции

SystemVerilog эффективно совмещает преимущества традиционного HDL (Verilog/VHDL) с гибкостью С-подобного синтаксиса и средствами

высокой степени автоматизации, что делает его идеальным для задач автономной верификации.

3.4.4.4 Сравнительный анализ инструментов верификации

В результате рассмотрения сравнительных характеристик языков описания аппаратуры, а также принимая во внимание данные по таблице 1.1 инструменты верификации, принято решение использовать VCS от Synopsys как основную платформу симуляции. Это решение обосновано следующими преимуществами:

- полная поддержка SystemVerilog и стандартов UVM;
- высокая скорость симуляции на больших проектах;
- поддержка функционального покрытия, assertions и верификационных IP.

3.4.4.4 Сравнительный анализ фреймворков верификации

Для организации среды автономной верификации рассмотрены фреймворки, приведенные в таблице 1.2.

Таблица 1.2 – Фреймворки для верификации

Фреймворк	Характеристика	Статус
OVM	Исторический стандарт от Mentor Graphics	Устарел, заменён UVM
VMM	Методология от Synopsys	Частично совместим, но не обеспечивает достаточную гибкость при работе
Cocotb	Python-библиотека для HDL-верификации	Удобна для простых задач, но не масштабируется под крупные проекты с SystemVerilog
UVM	Современный промышленный стандарт на основе SystemVerilog, поддерживает модульность, повторное использование, coverage-driven подход	Промышленный стандарт, поддержка последних тенденций в верификации

Выбор в пользу UVM обусловлен следующими причинами:

- стандартизация (IEEE 1800.2) — UVM является промышленным стандартом, поддерживаемым большинством симуляторов.
- гибкость — поддержка компонентов, таких как драйверы, мониторы, агенты, что упрощает читаемость и понимаемость кода;
- автоматизация — встроенная поддержка coverage, constrained random test generation, reusable verification environments;
- совместимость с VCS — нативная поддержка со стороны выбранной симуляционной платформы.

1.2.5 Постановка задачи на разработку

Проанализировав предметную область, решено, что, при реализации дипломного проекта будет необходимо создать систему автономной верификации, написанную на языке SystemVerilog, с использованием методологии UVM. Система должна запускаться при помощи VCS и обеспечивать верификацию BIST. Система должна включать в себя автоматическую сборку, автоматический запуск и прогон теста, а также систему сбора отчета о прохождении тестирования.

2 ОРГАНИЗАЦИОННОЕ ОБЕСПЕЧЕНИЕ ПРОЕКТА

2.1 Анализ вариантов решения проблемы исследования

Целью данного проекта является разработка тестовой системы для проверки правильности работы BIST. В нормальных условиях, устройство должно выявлять наличие ошибок памяти [18]. Поэтому для наиболее точной проверки необходимо симулировать работу памяти с ошибками. Есть несколько возможных подходов в данном случае:

- симуляция памяти прямым способом – через генерацию сигналов, в зависимости от входных данных;
- модернизация рабочей памяти для создания в ней ошибок;
- написание новой памяти с возможностью внедрения в нее ошибок.

В таблице 2.1 приведены преимущества и недостатки каждого метода.

Таблица 2.1 – Преимущества и недостатки вариантов эмуляции памяти

Тип эмуляции памяти	Преимущества	Недостатки
Прямой способ	Простота использования	Сложность реализации, необходимость следить за адресами обращения, необходимость выстраивания сложных временных последовательностей, необходимость копировать точное поведение памяти
Модернизация	Простота использования – ожидаемое поведение памяти	Сложность реализации, в некоторых случаях – полная невозможность реализации, сложность внедрения
Модернизация	Подразумевается с самого начала	Генерация случайных ошибок в SystemVerilog, сложность создания самих ошибок в уже готовом модуле
Создание новой	Простота реализации ошибок – скопировать временные параметры оригинальной памяти в отдельный модуль достаточно просто, простота использования, возможность генерации ошибок	Время разработки – необходимо с нуля писать новую память

Из всего вышеперечисленного можно сделать вывод, что самым удачным выбором для реализации данного проекта, будет являться написание новой памяти с механизмом внедрения ошибок.

Реализация такой памяти является нетривиальной задачей, особенно учитывая, что она должна интегрироваться с BIST, который работает по определенному протоколу. Среди вариантов реализации я рассматривал три варианта:

- использование чистого SystemVerilog;
- написание памяти на C++ и его интеграция в тестовую систему через “DPI-C”;
- написание памяти на C++ с интеграцией ошибок, реализованных на скриптовом языке.

Каждый из таких подходов был рассмотрен. В таблице 2.2 приведены преимущества и недостатки каждого метода, в таблице 2.3 приведены сравнения таких методов по некоторым критериям.

Таблица 2.2 – Преимущества и недостатки методов реализации памяти

Тип реализации	Преимущества	Недостатки
SystemVerilog	Простая интеграция с тестовой системой, статический контроль типов, удобность для симуляции и верификации	Ограниченная гибкость для сложной логики инъекций, сложность реализации динамических типов ошибок, низкая производительность при большом количестве логики
C++	Высокая производительность, можно использовать сложные структуры, удобно для повторного использования – нет привязки к конкретному проекту	DPI-C API требует точного соблюдения типов и соглашений, увеличивается сложность сборки, фиксированная инъекция ошибок на момент запуска и сборки – отсутствие возможности конфигурировать во время симуляции
C++ и скриптовый язык	Максимальная гибкость в настройке ошибок, можно загружать/переключать сценарии на лету, подходит для сложных/динамических ошибок, разделение логики памяти и тестирования	Самая высокая сложность реализации, необходимость добавить интерпретатор скриптового языка в C++, возможна потеря производительности при частом вызове скриптов

Таблица 2.3 – Сравнительная таблица реализации памяти

Критерий	SystemVerilog	C++	C++ и скрипты
Скорость симуляции	Средняя	Высокая	Высокая-средняя
Гибкость инъекций	Низкая	Средняя	Высокая
Простота реализации	Высокая	Средняя	Низкая
Расширяемость	Средняя	Средняя	Очень высокая
Зависимость от инструментов	Низкая	Средняя	Высокая

По приведенным выше данным для реализации проекта не подходит вариант реализации памяти с использованием чистого SystemVerilog. Хотя он и прост в реализации, но спектр его применения довольно узок.

В случае C++ и C++ со скриптами, наблюдается паритет – сложность реализации второй обеспечивает большую гибкость при работе, в то время как чистый C++ является средним по всем показателям. Для реализации выбран вариант создания памяти со скриптами – в проекте присутствует большой набор ошибок с разной степенью сложности, и их реализация является приоритетной задачей. На рисунке 2.1 представлена модель тестовой системы с использованием памяти со скриптами.

Модель тестовой системы

Модель тестовой системы для простого устройства BIST

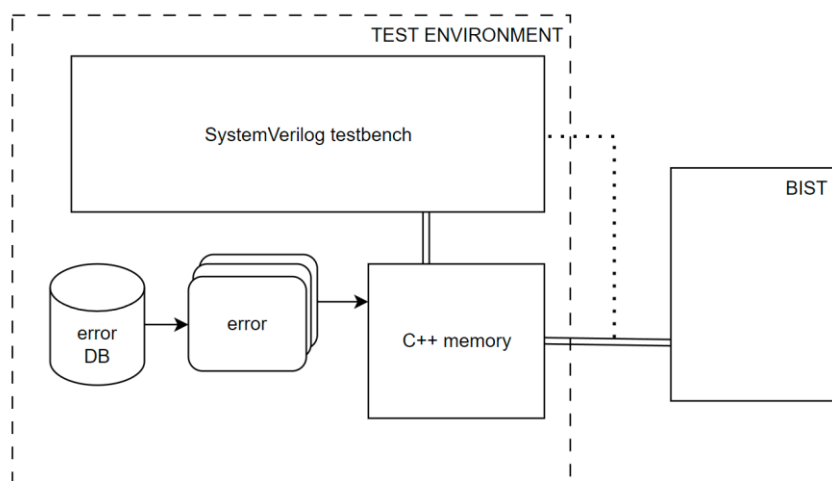
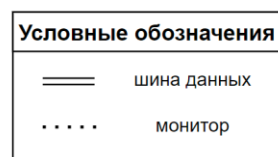


Рисунок 2.1 – Модель тестовой системы

В отдельных случаях модель будет усложняться. Например, в случае с памятью, в которой возможна замена ошибочных столбцов, дополнительно будет использоваться обычная память для эмуляции механизма смены столбца (рисунок 2.2). В таком случае до смены столбца устройство будет работать с памятью с ошибками, а смена столбцов и работа с исправленной памятью будет происходить уже с обычной памятью, т.к. считается, что после смены столбцов в памяти будут отсутствовать ошибки.

Модель тестовой системы для памяти с заменой

Модель тестовой системы для устройства BIST, поддерживающего механизм замены

Условные обозначения	
==	шина данных
.....	монитор

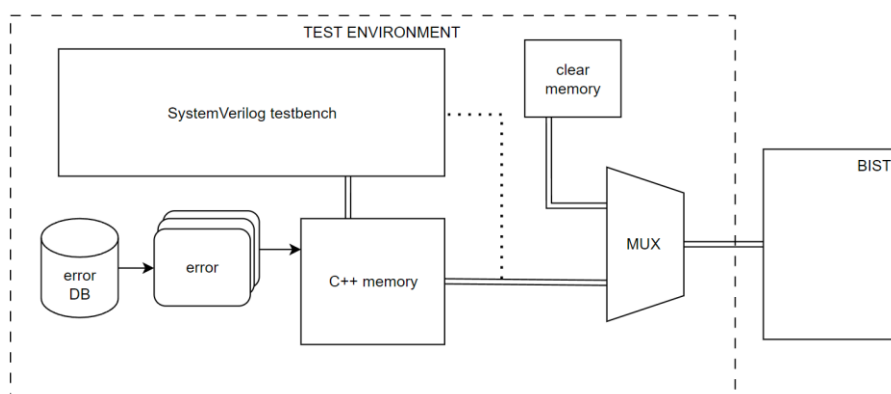


Рисунок 2.2 – Модель тестовой системы для памяти с заменой

2.2 Инфраструктура информационных технологий проекта

2.2.1 Системное и прикладное программное обеспечение

Информационные технологии, используемые в процессе реализации проекта, условно делятся на системное и прикладное программное обеспечение. Системное ПО обеспечивает базовую работу вычислительных ресурсов и инфраструктурных компонентов, тогда как прикладное программное обеспечение предназначено для выполнения прикладных задач

в области разработки, симуляции, анализа и сопровождения верификационной системы.

Системное программное обеспечение играет ключевую роль в обеспечении стабильной и предсказуемой работы среды разработки и верификации. В проекте выбрана операционная система Debian 8, т.к. на нем работают решения от Synopsys, которые в большинстве своем используются в данном проекте. Также в качестве вспомогательных системных инструментов используются:

- g++ – компилятор для сборки модулей на языке C++;
- GNU Make – средство автоматизации сборки;
- bash – оболочка для написания скриптов запуска, тестирования и анализа;
- lua – скриптовый язык, использующийся для работы скриптов ошибок;
- python 3.9 – используется как вспомогательное средство для генерации вариантов ошибок.

Прикладное ПО обеспечивает реализацию задач, непосредственно связанных с построением и функционированием автономной верификационной системы. Оно включает в себя средства разработки, симуляции, анализа покрытия и интеграции программных компонентов между собой.

В качестве основного средства разработки будет использоваться Visual Studio Code. С использованием плагинов, эта IDE является универсальной для работы с кодом, написанным на любом языке программирования.

Subversion (SVN) — система контроля версий, используемая в проекте для хранения, отслеживания изменений и организации совместной работы с исходным кодом. Несмотря на возрастающую популярность распределённых VCS, таких как Git, выбор в пользу SVN обоснован его простотой, централизованной моделью управления репозиторием, стабильной работой в

корпоративной среде, где необходим единый репозиторий и строгий контроль доступа, а также хорошей интеграцией с существующими инструментами разработки и управления проектами.

Основным инструментом симуляции и проведения верификации выступает промышленный симулятор Synopsys VCS, обеспечивающий:

- поддержку языка SystemVerilog с полным соответствием IEEE стандарту;
- интеграцию с UVM-средой;
- механизм DPI-C для взаимодействия с внешними модулями на языке C++;
- генерацию отчётов покрытия: functional, code (line, toggle, branch);
- поддержку формального моделирования и верификации с использованием SVA;
- взаимодействие с инструментом DVE для визуального анализа сигналов и волновых форм.

Для функционирования проекта используется как лицензионное, так и свободное ПО:

- Synopsys VCS, DVE, UVM — лицензионное, требует приобретения соответствующих прав;
- SVN, Python, GCC — распространяются под открытыми лицензиями (MIT, GPL и др.);

Используемый набор системного и прикладного программного обеспечения обеспечивает полную поддержку всех этапов разработки, тестирования и автономной верификации устройства. Комплексное использование VCS + UVM + C++ (DPI-C) позволяет строить надёжную и расширяемую инфраструктуру с возможностью масштабирования.

На рисунке 2.3 представлена схема сборки и компиляции проекта.

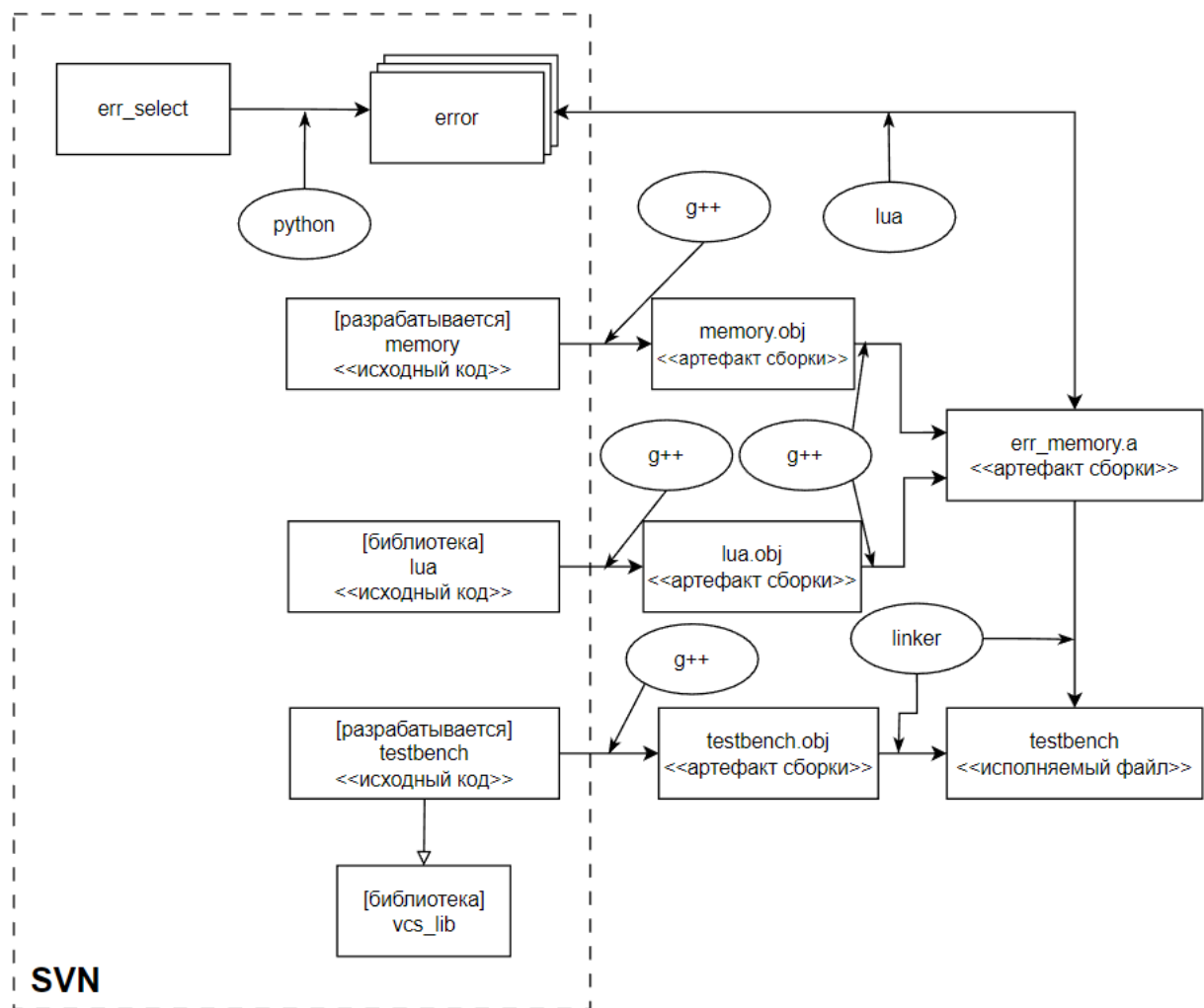


Рисунок 2.3 – Схема сборки и компиляции

На момент начала сборки в SVN находятся исходные коды для памяти – код памяти memory и код библиотек lua; скрипт селектора ошибок err_select; скрипты ошибок error, код тестовой системы testbench, использующей библиотеки VCS. Изначально собирается модель памяти. После сборки модели памяти, используя ее собирается основной исполняемый файл.

На этапе запуска программы для тестирования, запускается скрипт селектора ошибок, который подставляет выбранный скрипт ошибки в память.

2.2.2 Инструментальное программное обеспечение

В рамках реализации системы автономной верификации цифрового устройства была задействована совокупность библиотек и модулей, покрывающих следующие задачи:

- реализация UVM-инфраструктуры;
- автоматизация процесса симуляции и генерации отчётности;
- утилиты и обвязка на скриптовых языках (Bash, Python).

UVM — официальная стандартная библиотека от Accellera, предназначенная для построения модульной и переиспользуемой верификационной среды. Модули и компоненты, используемые в проекте:

- `uvm_component`, `uvm_env`, `uvm_test` – базовые классы;
- `uvm_driver`, `uvm_monitor`, `uvm_sequencer`, `uvm_agent` – структурные элементы тестовой системы;
- `uvm_config_db`, `uvm_factory`, `uvm_object_utils` – механизмы конфигурации и фабрик объектов;
- `uvm_report_server`, `uvm_verbosity` – отчетность и отладка;
- `uvm_sequence`, `uvm_sequence_item` – генерация транзакций;
- `uvm_tlm_analysis_fifo`, `uvm_analysis_port` – коммуникация между компонентами.

В контексте UVM существует несколько этапов, которые управляют потоком выполнения тестовой системы. Эти этапы обеспечивают стандартизированную структуру для проверки проектов с использованием UVM. Каждый этап представляет собой определенный этап процесса тестирования и служит определенной цели. Список этапов UVM:

- `build_phase`: эта фаза отвечает за создание и настройку компонентов тестовой системы. Она настраивает среду, создавая и подключая необходимые компоненты, такие как тест, DUT (Design Under Test) и

виртуальные интерфейсы. На этом этапе обычно создаются компоненты UVM и устанавливаются их конфигурации по умолчанию;

- `connect_phase`: на этом этапе устанавливаются соединения между различными компонентами тестовой системы. Это гарантирует, что соответствующие интерфейсы подключены к соответствующим портам тестируемого устройства. Этот этап особенно полезен при работе со сложными иерархиями тестовых систем или когда компонентам необходимо взаимодействовать друг с другом;

- `end_of_elaboration_phase`: эта фаза знаменует собой окончание создания тестовой системы. Это дает возможность выполнить любые окончательные настройки или проверки перед началом моделирования. Как правило, этот этап используется для проверки правильности выполнения всех необходимых подключений;

- `start_of_simulation_phase`: эта фаза выполняется непосредственно перед началом моделирования. Она позволяет компонентам выполнять любые необходимые задачи инициализации или настраивать мониторы, предикторы или сборщики покрытия.

- `run_phase`: это основная фаза, на которой происходит фактическое выполнение теста. Эта фаза содержит основную часть теста, включая генерацию стимулов, проверку ответов и сбор данных о покрытии. На этом этапе выполняется тестовая последовательность, которая запускает транзакции в DUT.

- `Extract_phase`: эта фаза является необязательной и используется для анализа конкретного теста или извлечения результатов. Это дает возможность проанализировать данные, собранные во время тестового прогона, или извлечь важную информацию для дальнейшего анализа или составления отчетов.

- `check_phase`: на этом этапе выполняются проверки правильности поведения тестируемого устройства. Она включает в себя сравнение

ожидаемых результатов с фактическими результатами, обнаружение ошибок или нарушений и отчет о статусе верификации.

- `report_phase`: эта фаза отвечает за создание окончательной сводки тестирования и любых других необходимых отчетов или логов. Она предоставляет информацию об общем ходе проверки, достигнутом покрытии и любых ошибках, обнаруженных во время теста.

На рисунке 2.4 представлен расширенный список фаз, с их иерархией.

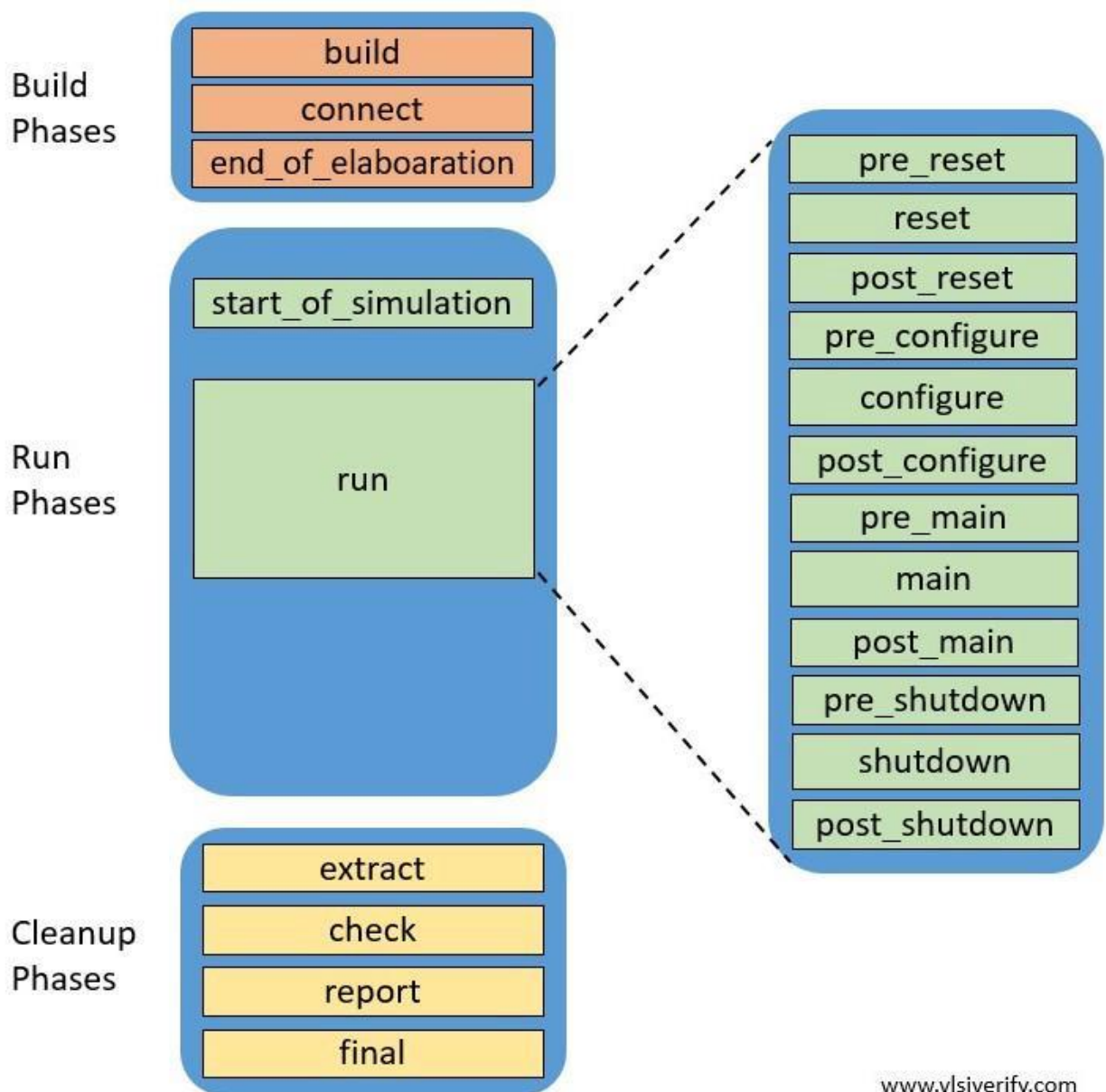


Рисунок 2.4 – Фазы UVM

Можно увидеть, что `run_phase` разбивается на двенадцать более мелких фаз. Первые три фазы условно относятся к фазам создания тестовой системы. Две следующие фазы относятся к основным, а заключающие — к фазам очистки. Так же на рисунке можно увидеть `final_phase`. Она служит для определения системой окончания тестирования и содержит в себе единственный метод завершения теста.

Выбор в пользу UVM обоснован его широкой индустриальной поддержкой, совместимостью с инструментами типа VCS и глубокой интеграцией с механизмами DPI и сбором покрытия.

Python используется для генерации конфигурационных файлов. Используемые библиотеки:

- `xml` – генерация файлов конфигурации;
- `random` – для случайной выборки скриптов ошибок;
- `sys` – для обработки параметров запуска.

Bash используется для компиляции и запуска, упаковки результатов.

2.2.3 Техническое обеспечение

Разработка исходного кода, написание верификационных тестов, подготовка конфигурационных файлов, скриптов и документации осуществляется на локальной машине — офисного типа. Эта машина не обладает высокой вычислительной производительностью, однако её ресурсов достаточно для редактирования исходных файлов и запуска предварительных проверок на синтаксическом уровне.

Характеристики локального ПК:

- Процессор – Intel Core i5-6500 (4 ядра, 3.2 ГГц);
- Оперативная память – 8 ГБ DDR4;

- Жесткий диск – HDD 256 ГБ;
- Графический адаптер – отсутствует дискретная видеокарта (используется встроенная графика Intel HD);
- Операционная система – Debian 11.

Такой тип конфигурации оптимален для работы с исходным кодом на языках SystemVerilog и C++, редактирования тестбенча и вспомогательных скриптов, подготовки отчётности и анализа логов, полученных после симуляции на сервере, а также взаимодействия с системой контроля версий SVN. Для взаимодействия с удалённым сервером на локальном ПК установлены:

- SSH-клиент;
- SCP – для обмена файлами;
- Visual Studio Code с плагином Remote SSH, позволяющим редактировать файлы на сервере напрямую.

Компиляция проектов на языке SystemVerilog, сборка C++ модуля, запуск симуляций в среде Synopsys VCS, а также выполнение нагрузочного тестирования производятся исключительно на выделенном удалённом сервере. Это связано с высокими требованиями к вычислительным ресурсам симулятора VCS, особенно при работе с UVM-инфраструктурой и при генерации отчётов покрытия.

Характеристики серверной платформы:

- Процессор – 2 × Intel Xeon Gold 6338 (64 ядра суммарно);
- Оперативная память – 256 ГБ DDR4 ECC;
- Хранилище – 2 ТБ NVMe SSD + 24 ТБ HDD;
- Операционная система – Debian 8;

Дополнительно на сервере установлено программное обеспечение:

- Synopsys VCS 2023.03;
- Python 3.9;
- GCC 11.2;

- Средства работы с отчётами покрытия (urg, dve, cov_merge, и др.)

На сервере реализованы полностью изолированная верификационная среда и централизованное хранилище с доступом по SSH. На рисунке 2.4 представлена модель работы системы.

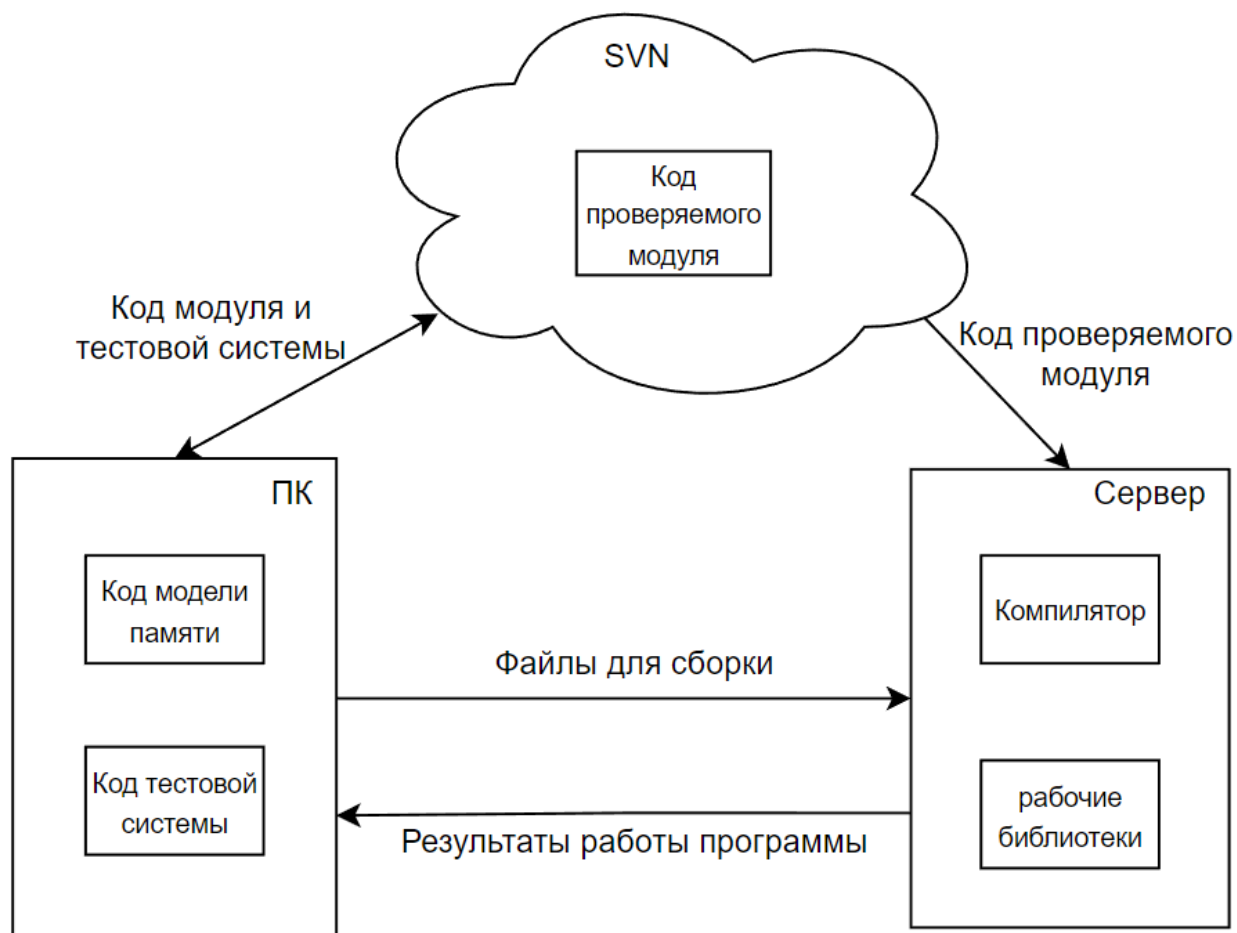


Рисунок 2.4 – Модель работы системы

Схема работы построена по клиент-серверной модели. Разработка и подготовка кода происходит локально. Через систему контроля версий или прямое копирование, код синхронизируется с сервером. Все ресурсоёмкие процессы в обязательном порядке запускаются на сервере. Результаты (логи, отчёты покрытия, временные диаграммы) переносятся обратно на локальную машину для анализа и визуализации. Такой подход позволяет эффективно

использовать вычислительные ресурсы, не перегружая рабочую машину, и масштабировать нагрузку при необходимости проведения массовых тестов.

2.3 Планирование реализации проекта

2.3.1 Резюме проекта

В рамках проекта была разработана система автономной верификации цифрового устройства BIST, ориентированная на встраиваемые и высоконагруженные вычислительные среды. Основная цель проекта – автоматизация процесса проверки корректности поведения устройства до этапа физической реализации, снижение вероятности функциональных ошибок и обеспечение воспроизводимости тестирования. Проект решает одну из ключевых задач современной цифровой схемотехники — эффективную и формализованную верификацию RTL-дизайна с использованием унифицированных методологий.

В рамках реализации приняты следующие проектные решения:

- использование языка SystemVerilog, обладающего богатым синтаксисом для описания тестовых сценариев, интерфейсов и assertions;
- применение индустриального фреймворка UVM что обеспечило модульность, расширяемость и повторное использование компонентов;
- интеграция внешнего функционального модуля, реализованного на C++, посредством интерфейса DPI-C, что позволило реализовать высокоуровневую функциональность (например, моделирование внешней среды или протоколов);

- реализация способа внедрения ошибок через внешние скрипты, что позволило увеличить гибкость в воссоздании ошибок и облегчить их использование во время работы системы;
- проведение симуляции и анализа покрытия с использованием Synopsys VCS, что обеспечило высокую точность и производительность.

Проектная система разделена на несколько слоёв:

- верификационная среда (testbench) на systemverilog;
- фреймворк UVM с реализацией драйверов, мониторов, генераторов и scoreboard;
- модуль C++, подключаемый по DPI, с использованием скриптов, имитирующий память с ошибками;
- конфигурационные файлы и скрипты автоматизации, обеспечивающие гибкую настройку и масштабирование тестов.

Рабочий процесс строится по модели «локальная разработка — удалённая симуляция». Исходный код пишется на офисной машине с базовыми характеристиками. Симуляции запускаются на сервере с высокой производительностью. Результаты анализируются через интерфейсы командной строки, лог-файлы и визуализацию (например, DVE или кастомные отчёты). В качестве системы контроля версий использована Subversion (SVN), обеспечивающая централизованное хранение и совместную разработку.

Результатом внедрения проектной системы является:

- повышение степени автоматизации и воспроизводимости процесса верификации;
- существенное сокращение времени на тестирование новых версий устройства;
- снижение вероятности ошибок, не выявленных до уровня RTL;
- облегчение поддержки, документирования и масштабирования верификационной среды.

2.3.2 Иерархическая структура работ проекта

Структура работ в рамках проекта по разработке системы автономной верификации устройства BIST включает шесть этапов. Ниже приведено текстовое описание WBS, сопровождаемое нумерацией и логическим делением. На рисунке 2.4 приведено графическое представление детализации этапов.

Основные этапы проекта:

- подготовительный этап;
- разработка верификационной среды;
- интеграция внешнего модуля;
- автоматизация симуляции;
- тестирование и отладка;
- документирование и завершение.
-

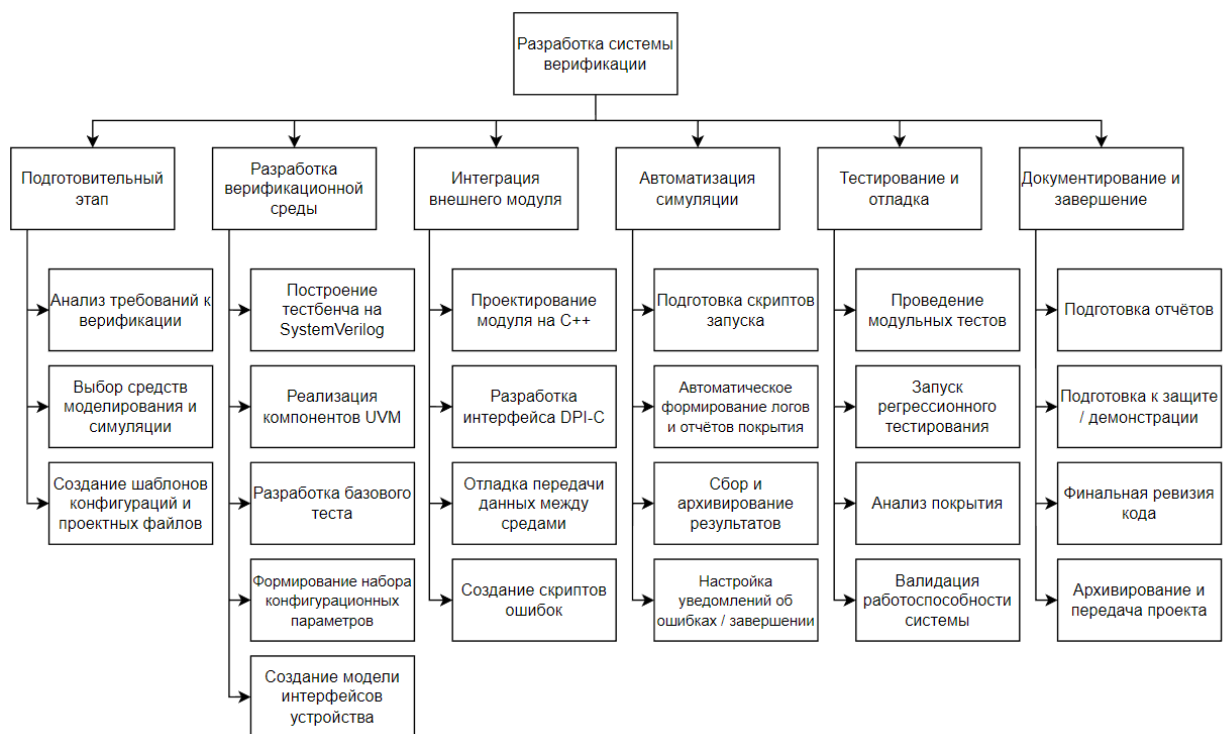


Рисунок 2.5 – Детализация этапов

- 1) Подготовительный этап:
 - а. Анализ требований к верификации.
 - б. Выбор средств моделирования и симуляции.
 - в. Создание шаблонов конфигураций и проектных файлов.
- 2) Разработка верификационной среды:
 - а. Построение тестбенча на SystemVerilog.
 - б. Реализация компонентов UVM.
 - в. Разработка базового теста.
 - г. Формирование набора конфигурационных параметров.
 - д. Создание модели интерфейсов устройства.
- 3) Интеграция внешнего модуля:
 - а. Проектирование модуля на C++.
 - б. Разработка интерфейса DPI-C.
 - в. Отладка передачи данных между средами.
 - г. Создание скриптов ошибок.
- 4) Автоматизация симуляции:
 - а. Подготовка скриптов запуска.
 - б. Автоматическое формирование логов и отчетов покрытия.
 - в. Сбор и архивирование результатов.
 - г. Настройка уведомлений об ошибках/завершении.
- 5) Тестирование и отладка:
 - а. Проведение модульных тестов.
 - б. Запуск регрессионного тестирования.
 - в. Анализ покрытия.
 - г. Валидация работоспособности системы.
- 6) Документирование и завершение.
 - а. Подготовка отчетов.
 - б. Подготовка к защите/демонстрации.
 - в. Финальная ревизия кода.
 - г. Архивирование и передача проекта.

Примечания к структуре:

- каждая задача привязана к срокам и ресурсам;
- некоторые работы выполняются параллельно (например, разработка SystemVerilog и C++ компонентов);
- WBS служит основой для построения сетевого графика, ресурсного плана и финансовой сметы проекта.

3.4.4 Оценка стоимости проекта

Оценка выполнялась на основании нормативных затрат времени и стоимости человеческих и вычислительных ресурсов. Проект носит исследовательско-инженерный характер, и его реализация велась в рамках корпоративной или университетской инфраструктуры, что позволило сократить расходы на программно-аппаратную платформу.

2.3.3.1 Затраты на трудовые ресурсы

Таблица 2.4 – Затраты на трудовые ресурсы

Вид работ	Исполнитель	Часы	Ставка (руб/ч)	Сумма (руб)
Анализ требований	Ведущий инженер	20	900	18.000
Разработка архитектуры testbench	Верификатор	120	800	96.000
Реализация компонентов UVM	Верификатор	120	800	96.000
Разработка C++ модуля	Верификатор	160	800	128.000
Интеграция DPI	Верификатор	40	800	32.000
Проведение тестирования и отладки	Верификатор	320	800	256.000
Генерация отчётов покрытия	Верификатор	40	800	32.000
Документирование	Руководитель проекта	20	1000	20.000
Итого				678.000

Наибольшую часть общей стоимости проекта, указанного в таблице 2.4, составляют трудозатраты специалиста, участвующего в проектировании, реализации и тестировании системы верификации. Это обосновано высокой квалификацией требуемого персонала, а также продолжительностью отдельных этапов разработки. Распределение задач позволило эффективно использовать ресурсы, минимизировав простои и дублирование усилий.

2.3.3.2 Затраты на программное обеспечение

Таблица 2.5 – Затраты на программное обеспечение

ПО	Тип лицензии	Примечание	Стоимость (руб)
Synopsys VCS	Корпоративная	Используется на сервере	0
Subversion	Open Source	Контроль версий	0
Python	Open Source	Скрипты и анализ логов	0
Bash tools	Open Source	Автоматизация сборки	0
g++	Open Source	Для компиляции DPI модуля	0
Microsoft Office	Корпоративная	Подготовка отчетов	5.000
Итого			5.000

Поскольку в проекте использовались преимущественно open source решения и уже лицензированные коммерческие средства (лицензия VCS предоставлена на сервере центра разработки, не входит в прямые затраты проекта), прямые затраты на программное обеспечение оказались минимальными. Это повысило экономическую эффективность проекта и сделало его более доступным для дальнейшего масштабирования или переноса в другие среды.

2.3.3.3 Затраты на технические ресурсы

Таблица 2.6 – Затраты на технические ресурсы

Ресурс	Описание	Ставка/мес (руб)	Доля использования	Период (мес)	Сумма
Рабочее место	ПК	15.000	100%	5	75.000
Сервер симуляции	Выделенный сервер	80.000	20%	5	80.000
Сетевые ресурсы	Хранилище, SVN	10,000	90%	5	45.000
Итог					200.000

Проект не потребовал значительных расходов (таблица 2.6) на аппаратные средства, так как основная нагрузка по симуляции и анализу ложилась на уже имеющийся мощный сервер. Использование офисного ПК для написания кода и выполнения вспомогательных операций позволило дополнительно сократить затраты на оборудование.

2.3.3.4 Прочие расходы

К прочим расходам можно отнести резерв непредвиденных затрат, который будет составлять 5% от общей стоимости проекта.

2.3.3.5 Сводная таблица оценки стоимости

Таблица 2.7 – сводная таблица оценки стоимости

Категория затрат	Сумма (руб)
Трудовзатраты	678.000
Программное обеспечение	5.000
Технические ресурсы	200.000
Сумма затрат	883.000
Резерв, 5%	44.150
Итог	927.150

Итоговая стоимость проекта составила около 927 тысяч рублей, что является оправданным уровнем вложений для инженерно-исследовательского проекта подобного типа. Благодаря рациональному использованию ресурсов,

применению существующих инструментов и перераспределению задач между специалистами удалось обеспечить высокое качество результата при сравнительно умеренном бюджете.

3.4.4 Расписание проекта

Проект по разработке и верификации устройства встроенного самотестирования памяти условно делится на 8 основных этапов.

1. Анализ требований включает в себя: анализ требований к верификации, создание документации на оборудование для верификации.
 - а. Продолжительность: 1 неделя.
 - б. Участники: ведущий инженер.
2. Разработка архитектуры тестовой системы включает в себя: разработку интерфейсов согласно документации.
 - а. Продолжительность: 3 недели.
 - б. Участники: верификатор.
3. Реализация компонентов UVM включает в себя: создание драйверов, мониторов и чекеров под устройство.
 - а. Продолжительность: 3 недели.
 - б. Участники: верификатор.
4. Разработка C++ модуля включает в себя: разработку модели неисправной памяти, разработку скриптов ошибок.
 - а. Продолжительность: 4 недели.
 - б. Участники: верификатор, разработчики.
5. Интеграция DPI включает в себя: интеграция C++ модуля в тестовую систему.
 - а. Продолжительность: 1 неделя.
 - б. Участники: верификатор.

6. Проведение тестирования и отладки включает в себя: разработку тестовой базы, проведение тестов, отладку найденных багов.
 - а. Продолжительность: 8 недель.
 - б. Участники; верификатор, ведущий инженер.
7. Генерация отчетов покрытия включает в себя: сбор покрытия кода, анализ покрытия, создание исключений.
 - а. Продолжительность: 1 неделя.
 - б. Участники: верификатор.
8. Документирование включает в себя: обобщение проведенного тестирования, создание базы отчетов покрытия.
 - а. Продолжительность: 1 неделя.
 - б. Участники: верификатор, руководитель проекта.

Наглядные сроки можно увидеть в таблице 2.8.

Таблица 2.8 – Расписание проекта

Этап	Продолжительность	Сроки
Анализ требований	1 неделя	03.06.2024–10.06.2024
Разработка архитектуры testbench	3 недели	10.06.2024–01.07.2024
Реализация компонентов UVM	3 недели	01.07.2024–22.07.2024
Разработка C++ модуля	4 недели	22.07.2024–19.08.2024
Интеграция DPI	1 неделя	19.08.2024–26.08.2024
Проведение тестирования и отладки	8 недель	26.08.2024–21.10.2024
Сбор отчетов покрытия	1 неделя	21.10.2024–28.10.2024
Документирование	1 неделя	28.10.2024–04.11.2024

Разработанное расписание проекта обеспечивает структурированный подход к процессу верификации и интеграции компонентов, позволяя эффективно контролировать сроки выполнения задач и оптимально распределять ресурсы. Восемь основных этапов, начиная с анализа требований и заканчивая документированием, охватывают все ключевые стадии создания и отладки системы.

Наиболее длительным и ресурсоемким этапом является тестирование и отладка (8 недель, с 26.08.2024 по 21.10.2024), поскольку на данном этапе происходит всесторонняя проверка взаимодействия компонентов, включая C++ модуль и UVM-структуру, а также устранение выявленных ошибок. Разработка C++ модуля (4 недели) и реализация компонентов UVM (3 недели) также требуют значительных усилий, поскольку они формируют основу всей системы.

Этапы анализа требований и интеграции DPI, несмотря на их относительно короткую продолжительность (по 1 неделе), являются критически важными для правильной постановки задач и корректной стыковки всех модулей. Разработка архитектуры тестовой системы (3 недели) играет важную роль в обеспечении корректной среды для верификации, а этапы сбора отчетов покрытия и документирования (по 1 неделе каждый) завершают проект, обеспечивая полноту отчетности и передачу результатов.

Представленное расписание, охватывающее период с 03.06.2024 по 04.11.2024, выстроено с учетом логической последовательности работ и четкого распределения обязанностей между участниками проекта. Такой подход позволяет реализовать проект в ограниченные сроки без ущерба для качества, что особенно важно для верификационных систем, предъявляющих высокие требования к надежности. При необходимости график может быть скорректирован в ходе реализации, но уже на этапе планирования он служит прочной основой для эффективного управления временем и ресурсами.

2.3.5 Формирование календарно-ресурсного плана проекта

Подробная диаграмма и календарно-ресурсная модель приведены на рисунке 2.6. Важно отметить, что в процессе реализации проекта план может подвергаться уточнениям, особенно в случае изменений требований или

появления новых задач, связанных с обеспечением отказоустойчивости и масштабируемости системы управления онлайн-заказами.

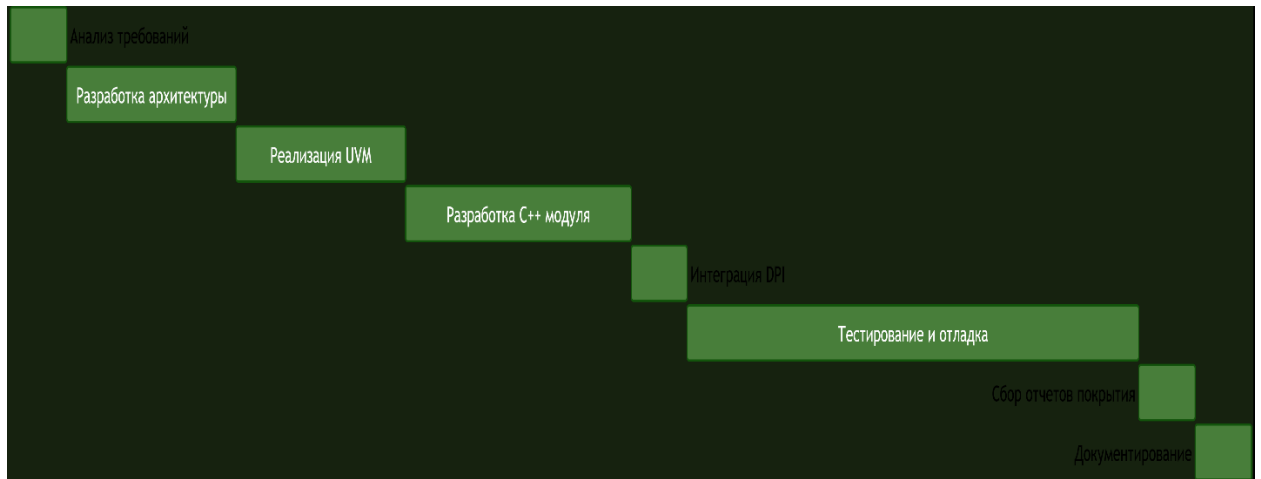


Рисунок 2.6 – Диаграмма Ганта

Такой подход обеспечивает наглядность текущего статуса проекта, способствует своевременному выявлению рисков срыва сроков и позволяет более гибко управлять ресурсами на протяжении всего жизненного цикла проекта.

3 РЕЗУЛЬТАТЫ ПРОЕКТИРОВАНИЯ ИНФОРМАЦИОННОЙ СИСТЕМЫ

3.1 Сценарии работы пользователей

3.1.1 Варианты использования системы

Система предполагает интеграцию с инфраструктурой верификации RTL-дизайна, где основными действиями являются запуск тестов, сбор результатов симуляции, анализ покрытия и ведение отчётности. В таблице 3.1 приведены акторы и их роль в системе. На рисунке 3.1 приведена схема use-case.

Таблица 3.1 – Участники

Актор	Роль в системе
Инженер по верификации	Создаёт и запускает тесты, анализирует покрытие
Разработчик RTL-дизайна	Проверяет корректность тестов, отслеживает ошибки
Система симуляции (VCS)	Выполняет тесты и генерирует выходные данные
SVN	Управляет версиями тестов и конфигураций

Основные сценарии:

- запуск симуляции тестов вручную;
- добавление нового тестового сценария;
- анализ покрытия кода;
- получение отчёта о результатах тестирования;
- интеграция DPI-модуля на C++

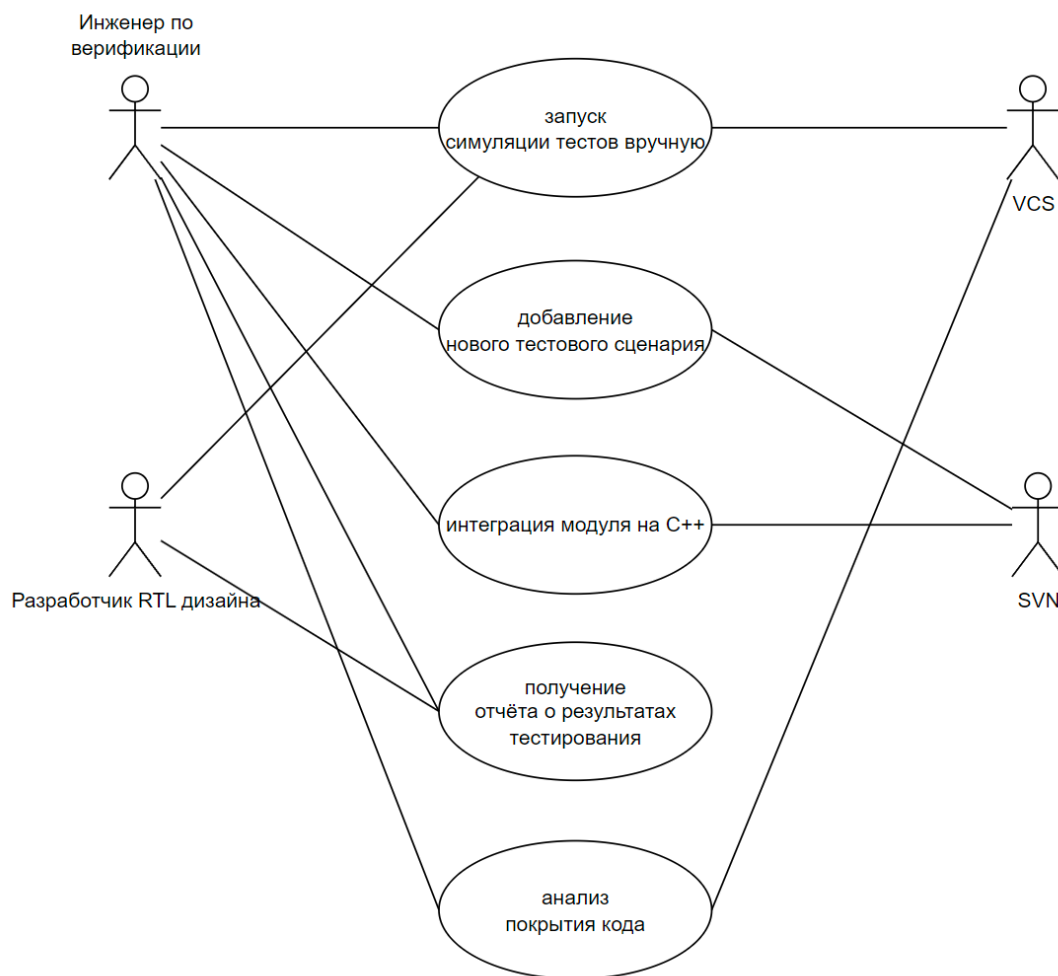


Рисунок 3.1 Use-case диаграмма.

Описание ключевых сценариев

Сценарий 1 – ручной запуск симуляции. Акторы – VCS, инженер по верификации. Цель – проверить модуль вручную. Предусловия – исходный код и тестовая система синхронизированы из SVN. Последовательность

- инженер подготавливает конфигурационный файл для симуляции;
- через CLI отправляет команду на запуск теста на сервере;
- сервер запускает симулятор VCS;
- результаты сохраняются в логах и формате покрытия;
- инженер анализирует результаты и принимает решение о доработке.

Сценарий 2 — добавление нового теста. Акторы – SVN, инженер по верификации. Цель – расширить покрытие верификации. Предусловия – тестовая система готова, базовая архитектура настроена. Последовательность:

- верификатор создаёт новый тестовый класс на SystemVerilog или новый скрипт ошибки;
- регистрирует тест в конфигурационном списке;
- коммитит изменения в SVN;
- тест автоматически включается в регрессию.

Сценарий 3 – интеграция модуля на C++. Акторы – SVN, инженер по верификации. Цель – расширить функциональность тестовой системы. Последовательность:

- верификатор пишет C++ модуль;
- создаёт интерфейс через DPI-C и описывает связывающие блоки в SystemVerilog;
- собирает симуляцию с ключами -dpi и -f;
- добавляет тест в конфигурационный список;
- коммитит изменения в SVN.

Сценарий 4 – анализ покрытия кода. Акторы – VCS, инженер по верификации. Цель – оценить полноту верификации. Последовательность:

- после симуляции VCS сохраняет файл покрытия (vdb);
- инженер открывает его в DVE или отчетной утилите;
- выявляет непокрытые участки, принимает решение о создании дополнительных тестов.

Сценарий 5 – получение отчета о результатах тестирования. Акторы – разработчик RTL дизайна, инженер по верификации. Цель – получить отчет о результате тестов. Последовательность:

- после каждого запуска формируется текстовый или HTML-отчёт;
- отчёты хранятся на сервере или передаются по сети;
- пользователь открывает лог/отчет и анализирует результат.

Рассмотренные варианты использования демонстрируют гибкость и адаптируемость системы автономной верификации в различных рабочих ситуациях. Наличие как ручных, так и автоматических сценариев позволяет интегрировать систему как в исследования, так и в производственные процессы.

3.1.2 Формы документов

В рамках проекта предусматриваются следующие основные виды документов: конфигурационные файлы, тестовые отчеты, отладочные логи, отчеты покрытия, документы сопровождения.

3.1.2.1 Конфигурационные файлы

Конфигурационные файлы представляют собой текстовые документы, содержащие параметры запуска симуляции и компиляции. Эти файлы необходимы для корректной настройки среды верификации, включая подключение нужных модулей, указание параметров компиляции, определение точек входа и путей к дополнительным библиотекам.

В проекте используются простые `xml` и `cfg` форматы, которые описывают конкретный тест, набор параметров и флаги компиляции для симулятора Synopsys VCS. Также могут быть задействованы `svh`-файлы, в которых размещаются директивы препроцессора и макросы UVM. Благодаря использованию таких конфигурационных файлов, обеспечивается модульность и повторяемость тестов, что критически важно в системах автономной верификации. В листинге 3.1 приведен пример конфига для модели памяти.

Листинг 3.1 – Конфиг для модели памяти

```
<fault_description>
  <gen_tools>
    <file name="/bist
/model/cmodel/src/fault_models/FaultGenerator.lua" />
  </gen_tools>
<cell_array_faults>
  <faults_binding hier_module_name="bist_tb_top.dut.mem">
    <concrete_fault name="c">
      <role name="victim" row="89" column="12" />
    </concrete_fault>
  </faults_binding>
  <abstract_model name="StuckAtFault">
    <file name="
/bist/model/cmodel/src/fault_models/StuckAtFault.lua" />
  </abstract_model>
  <concrete_model name="c">
    <base_abstract_model name="StuckAtFault" />
    <random_parameter name="value">
      <item value="0" weight="1" />
      <item value="1" weight="1" />
    </random_parameter>
  </concrete_model>
</cell_array_faults>
</fault_description>
```

3.1.2.2 Форма тестового отчета

Результаты работы системы автономной верификации оформляются в виде отчётных документов, фиксирующих ход и итог выполнения теста. Основные формы представляют собой текстовые или HTML-отчёты, которые автоматически формируются после завершения симуляции. Они включают в себя наименование теста, время запуска, параметры конфигурации, информацию о количестве успешно пройденных и проваленных проверок, а также диагностические сообщения.

Отчёты являются ключевым элементом обратной связи для инженера по верификации, позволяя оценивать текущее состояние DUT (design under test) и при необходимости — адаптировать UVM-стимулы. Эти документы автоматически архивируются и сохраняются в общей системе хранения

данных, что облегчает аудит и ретроспективный анализ. Пример тестового отчета приведен в листинге 3.2.

Листинг 3.2 – Тестовый отчет

StAlone run results for BIST , launched at 24-08-15_10-01 XML description of current launch: ../bist/progon.xml Report files: ../BIST/24-08-15_10-01/ Logs of the failed tests: ../BIST/24-08-15_10-01/failed_logs/ Time spent: 9 min.		
Revisions: RTL: 48062; testbench: 47889		
VCS version: No version found		
Tests run: 110		
Tests failed: 0		
Tests passed: 110		
Testname		Seed
Host Runtime (h:m:s)		
bist_1p_saf_test (MEM_1P) (1)		4846
irc-015 00:00:46		
bist_1p_sof_test (MEM_1P) (1)		1567
irc-015 00:00:45		
bist_1p_sf_test (MEM_1P) (1)		7804
irc-015 00:00:45		
bist_1p_wdf_test (MEM_1P) (1)		9042
irc-015 00:00:45		
bist_1p_rdf_test (MEM_1P) (1)		2204
irc-015 00:00:45		
.		
.		
.		
bist_1p_wdf_test (WRAPPER_E_R_JTAG) (1)		8798
irc-118 00:01:19		
bist_1p_rdf_test (WRAPPER_E_R_JTAG) (1)		6047
irc-045 00:01:09		
bist_1p_drf_test (WRAPPER_E_R_JTAG) (1)		48
irc-140 00:00:58		
bist_1p_pf_test (WRAPPER_E_R_JTAG) (1)		7704
irc-338 00:01:02		
bist_1p_cfdr_test (WRAPPER_E_R_JTAG) (1)		9047
irc-187 00:01:08		
bist_1p_cfds_test (WRAPPER_E_R_JTAG) (1)		6324
irc-308 00:01:07		
bist_1p_cfid_test (WRAPPER_E_R_JTAG) (1)		8914
irc-332 00:00:58		
bist_1p_cfin_test (WRAPPER_E_R_JTAG) (1)		748

Продолжение листинга 3.2 – Тестовый отчет

irc-300	00:01:23	
bist_lp_cfst_test (WRAPPER_E_R_JTAG) (1)		9459
irc-382	00:04:40	
bist_lp_cftr_test (WRAPPER_E_R_JTAG) (1)		5872
irc-287	00:01:25	
bist_lp_cfwd_test (WRAPPER_E_R_JTAG) (1)		697
irc-309	00:01:00	

3.1.2.3 Журналы логирования и отладочные логи

Журналы логирования служат важнейшим инструментом анализа поведения системы во время симуляции. Они фиксируют весь ход компиляции, симуляции, взаимодействие с DPI-модулями и другие события, происходящие в процессе выполнения. В случае сбоя или получения некорректных результатов, инженер может использовать отладочные логи для детального изучения причины возникновения ошибки.

Формат журналов представляет собой обычные текстовые log и txt файлы, создаваемые средствами симулятор. В проекте предусмотрена автоматическая генерация логов при каждом запуске теста, их архивация и индексирование для удобства последующего анализа. Пример отладочного лога приведен в листинге 3.3.

Листинг 3.3 – Отладочные логи

Notice: timing checks disabled with +notimingcheck at compile-time Chronologic VCS simulator copyright 1991-2018 Contains Synopsys proprietary information. Compiler version O-2018.09-SP1_Full64; Runtime version O-2018.09-SP1_Full64; Sep 23 16:21 2024 12354 22-04 Base model StuckAtFault wase loaded to bist_tb_top.dut.mem module successfully.
*****NOW DEFINED*****
*****MEM_1P*****
*****Verification of bist_mem_1p.v*****

Продолжение листинга 3.3 – Отладочные логи

```
UVM_INFO @ 0: reporter [RNTST] Running test bist_lp_saf_test...
NOTE: automatic random seed used:          1
Event a triggered
UVM_INFO @ 0.000 ns: reporter [TIMOUTSET]
'+UVM_TIMEOUT=800_000_000,' provided on the command line is
being applied.
UVM_INFO @ 0.000 ns: reporter [MAXQUITSET]
'+UVM_MAX_QUIT_COUNT=25' provided on the command line is being
applied.
Left = 0, right = 1
UVM_INFO
../uvm/environments/bist_lp_env/bist_lp_scoreboard.sv(103) @
15605.000 ns: uvm_test_top.env.scoreboard [[scoreboard]]
Everything is fine
check phase
report phase start
UVM_INFO ../uvm/tests/bist_base_test.sv(52) @ 15605.000 ns:
uvm_test_top [bist_lp_saf_test]
```

```
-----TEST PASSED-----
```

```
report phase end
```

```
UVM_INFO /uvm/uvm-1.2/src/base/uvm_report_server.svh(847) @
15605.000 ns: reporter [UVM/REPORT/SERVER]
--- UVM Report Summary ---
Quit count :      0 of      25
** Report counts by severity
UVM_INFO :      5
UVM_WARNING :      0
UVM_ERROR :      0
UVM_FATAL :      0
** Report counts by id
[MAXQUITSET]      1
[RNTST]           1
[TIMOUTSET]       1
[[scoreboard]]    1
[bist_lp_saf_test] 1

$finish called from file "/uvm/uvm-1.2/src/base/uvm_root.svh",
line 517.
$finish at simulation time 15605.000 ns
      V C S   S i m u l a t i o n   R e p o r t
Time: 15605000 ps
CPU Time:      0.540 seconds;      Data structure size:   0.4Mb
Thu Sep 23 16:21:53 2024
```

3.1.2.4 Отчёты покрытия (Coverage Reports)

Оценка полноты верификации невозможна без формирования отчётов покрытия. В данной системе отчёты покрытия создаются после выполнения симуляции и позволяют количественно оценить, насколько полно были проверены отдельные конструкции кода. Используются как стандартные метрики (score, branch, line, condition), так и пользовательские covergroup-наблюдения, определяемые в UVM-компонентах.

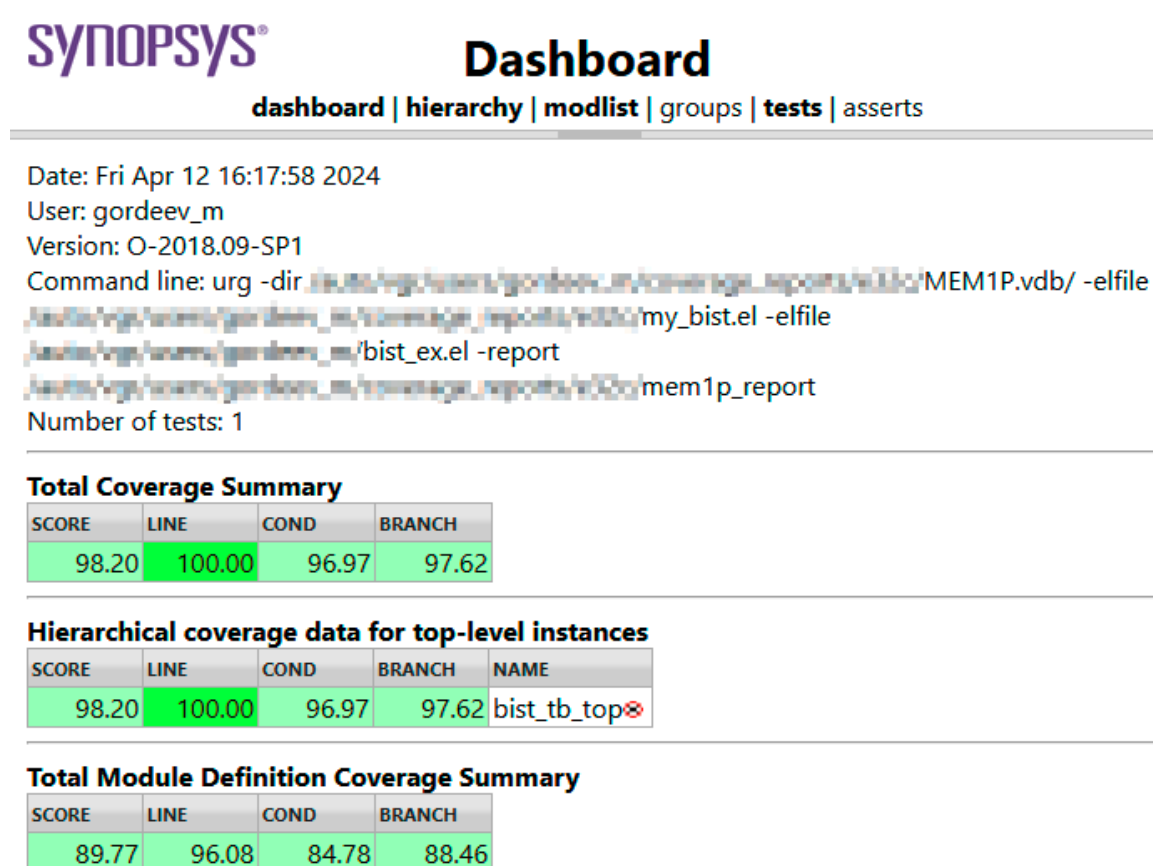


Рисунок 3.2 – Отчет покрытия

Инструментом формирования отчётов выступает утилита urg, входящая в состав Synopsys VCS. Итоговые отчёты предоставляются в виде HTML-файлов, пригодных для визуального анализа, и бинарных баз (vdb) для загрузки в специализированные средства просмотра, такие как Verdi или DVE. Эти отчёты используются для принятия решений о необходимости

расширения тестов или выявления непокрытых сценариев. Пример отчета покрытия приведен на рисунке 3.2.

3.1.2.5 Документы сопровождения

В рамках проекта также разрабатываются документы, предназначенные для поддержки пользователей и разработчиков системы верификации. Они включают:

- Инструкции по запуску симуляций, в которых пошагово описаны действия пользователя, начиная от получения последней версии кода до анализа покрытия;
- Архитектурное описание системы, содержащее информацию о логике работы окружения, структуре тестов и принципах взаимодействия между компонентами;
- Шаблоны тестов и конфигураций, позволяющие стандартизировать процесс разработки и ускорить создание новых верификационных блоков.

Сборка и запуск для

```
export PATH=$PATH:$HOME/bin
make -f $HOME/.vscode/extensions/ms-vscode.cortex-debug/Makefile run TESTNAME=<TestName> [GUI=ON] PROJECT=<Project> DUT=<Имя тестируемого устройства>
```

Тестируемые устройства:
MEM_1P - используется для тестирования небольших однопортовых памятей
MEM_1P_DU - используется для тестирования однопортовых памятей с возможностью установки dontuse-битов
WRAPPER - используется для тестирования однопортовых памятей с возможностью установки dontuse-битов и замены столбцов механизмом column-redundancy
WRAPPER_E_R - используется для тестирования однопортовых памятей с возможностью установки dontuse-битов, получения настроек column-redundancy из памяти (E)FUSE, их дополнения и активации
MEM_1P_DU - используется для тестирования двухпортовых памятей с возможностью установки dontuse-битов
KERNEL - Адро BIST'08

Описание тестовых сценариев и тестов

Summary	TestName	Description	Test done	Test passed
Базовый тест и проверка сборки. Доступные устройства: ANY				
Базовый тест	bist_base_test	Проверка сборки DUT и окружения. Для KERNEL является основным тестом. Критерии прохождения тестов: нет ошибок компиляции и uvm_fatal. для KERNEL: последовательность соответствует документации	Done	r48062
Базовый тест	bist_mem_test	Проверка сборки модели памяти. Критерии прохождения тестов: нет ошибок компиляции и uvm_fatal.	Done	r48062
Тесты однопортовых памятей. Доступные устройства: MEM_1P, MEM_1P_DU, WRAPPER, WRAPPER_E_R				
Тест случайной ошибки	bist_1p_rnd_test	Тест, запускающий случайный тип ошибки Критерии прохождения тестов: MEM_1P - устройство выдает сигнал ошибки в правильный момент MEM_1P_DU - после окончания работы устройства, проверяется правильность выставления данных в банке don't use WRAPPER - проверяется правильность передачи данных по каналу red_data WRAPPER_E_R - проверяется правильность передачи данных по каналу red_data 3 раза: первый и третий раз проверка правильности работы red_mem, второй раз проверка правильности выставления ошибочного столбца	Done	r48062
Тест идеальной памяти	bist_1p_ideal_test	Тест, запускающий идеальную память, не содержащую ошибок Критерии прохождения тестов: по окончании проверки ни разу не был зарегистрирован сигнал ошибки	Done	r48062
Тест определенной ошибки одной ячейки	bist_1p_saf_test bist_1p_sof_test bist_1p_tf_test bist_1p_vdf_test bist_1p_vdf_test	Тест, запускающий выбранный тип ошибки Критерии прохождения тестов: MEM_1P - устройство выдает сигнал ошибки в правильный момент MEM_1P_DU - после окончания работы устройства, проверяется правильность выставления данных в банке don't use WRAPPER - проверяется правильность передачи данных по каналу red_data	Done	r48062

Рисунок 3.3 – Документация на систему

Эти документы составляют основу для передачи знаний, стандартизации рабочих процессов. Пример документации приведен на рисунке 3.3.

3.2 Информационное обеспечение

3.2.1 Выбор и обоснование архитектуры

Эффективная разработка тестовой системы начинается с определения основных компонентов и транзакционных классов. Важно сформировать общую модель окружения и продумать взаимодействие каждого элемента с соответствующими транзакциями. Это требует глубокого анализа спецификации устройства и понимания его принципов работы.

На начальном этапе проводится анализ используемых интерфейсов, после чего разрабатываются агенты для каждого из них и создаются соответствующие транзакционные классы. В рамках моего тестового окружения были описаны агенты для разных вариантов сборок устройства:

- bist_1p_agent
- bist_1p_du_agent
- bist_wrapper_agent
- bist_wrapper_e_r_agent

В каждый из агентов входит driver, monitor, sequencer.

Драйвер интерфейса выполняет две параллельные функции:

- Сброс интерфейса – при получении сигнала reset все поля интерфейса обнуляются;
- Передача данных – драйвер проверяет наличие транзакций от генератора. Если транзакция получена, её поля записываются в соответствующие сигналы интерфейса (в соответствии со

спецификацией), после чего генератору отправляется подтверждение.

Классы транзакций определяют поля необходимой разрядности, соответствующие шинам (адреса, данные, размер пакетов, параметры бёрстов и т. Д.). Мониторы в агентах фиксируют состояние интерфейсов и передают данные в анализатор и хранилище, что требуется для проверки правильности работы устройства.

На рисунке 3.2 представлена блок-схема тестовой системы с точки зрения UVM окружения.

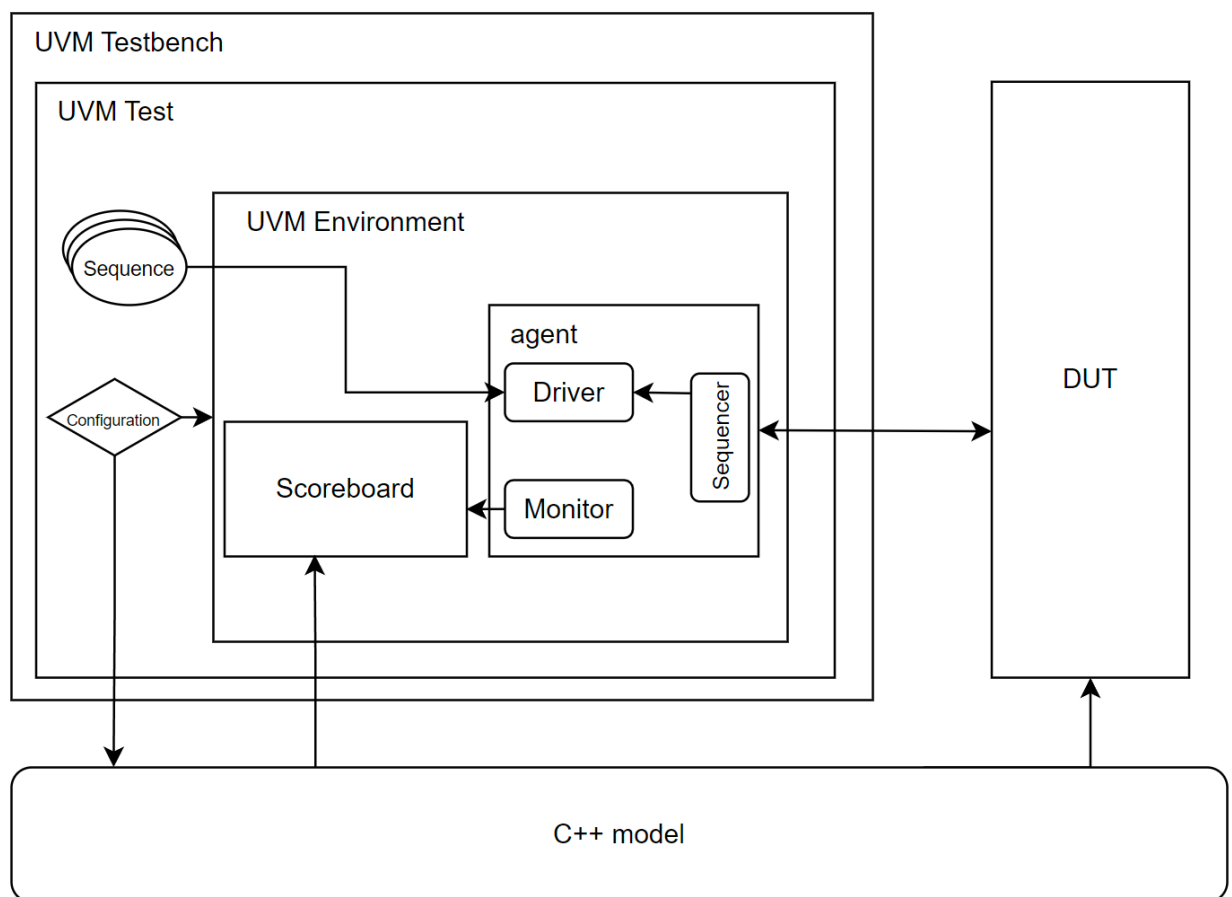


Рисунок 3.2 – Блок-схема тестовой системы

3.2.1.1 Разработка анализатора и алгоритма проверки данных

В процессе автономной верификации любого цифрового устройства анализатор играет важную роль в тестовом окружении. Этот компонент, с

помощью функций и алгоритмов, определенных в нем, обеспечивает возможность более качественного и всестороннего тестирования, такого как детальная проверка данных транзакций, проверка порядка, соответствие ограничениям и контроль количества запросов. Более того, все это происходит автоматически, что позволяет проводить множество повторных тестов на большом объеме транзакций.

В рамках работы по верификации устройства встроенного самотестирования памяти анализатором является scoreboard. Для различных вариантов работы проверяемого устройства, в нем представлено 3 варианта проверки правильности;

1. Прямая проверка. По спецификации работы устройства, оно должно передавать сигнал ошибки в момент ее обнаружения. Scoreboard отлавливает этот сигнал и сверяется с конфигурационными данными – адресом и номером ошибочной ячейки. Если сигнал ошибки не появился в правильное время, scoreboard засчитывает промах устройству.
2. Проверка выставления don't use битов. В случае работы с don't use памятью, устройство должно при нахождении ошибки записывать ее адрес и номер ячейки в отдельную память don't use. Если записи не было, или она была проведена по неправильному адресу, scoreboard засчитывает промах устройству.
3. Проверка redundancy. В случае работы с redundancy-памятью, после полного прохода устройство должно заменить неисправные столбцы. В момент замены, scoreboard перехватывает эти данные. Если замена не была произведена или была произведена по неправильному адресу, scoreboard засчитывает промах устройству.

3.2.1.2 Интеграция модели C++

Модель памяти с возможностью внедрения ошибок используется для эмуляции работы памяти и проверки устойчивости DUT к ошибочным состояниям. Она располагается на одном уровне с DUT и взаимодействует с тестовым окружением через интерфейсы SystemVerilog.

Модель памяти находится на том же уровне, что и DUT, и подключается к тестовой среде через интерфейс.

Принципы работы:

- Инициализация модели памяти. Модель памяти на C++ создаётся как отдельный класс, содержащий буфер данных, методы чтения и записи, методы инъекции ошибок;
- Связь UVM и C++ через DPI/SystemC;
- Интеграция в UVM Testbench;
- Внедрение ошибок и управление конфигурацией;
- Верификация устойчивости DUT.

Такой подход позволяет эффективно тестировать устойчивость DUT к ошибкам памяти. В листинге 3.4 приведен пример модуля с подключенной моделью памяти. В примере опущена большая часть кода для краткости.

Листинг 3.4 – Пример модуля с моделью

```
module dti_1pr_tm16_128x33_2ww1x_m_a (  
  VDD,  
  VSS,  
  DO,  
  A,  
  DI,  
  CE_N,  
  GWE_N,  
  T_RWM,  
  T_DLY,  
  CLK  
);  
import "DPI-C" context latchInputs_1prf = function int  
latchInputs_inner(/*input bit clock_en,*/ input logic global_we,  
input logic [addr_width-1:0] addr, input logic [data_width-1:0]  
data_in);
```

Продолжение листинга 3.4 – Пример модуля с моделью

```
import "DPI-C" context executeCommands_1prf = function int
executeCommands_inner();
import "DPI-C" context measureOutput_1prf = function int
measureOutput_inner(output logic [data_width-1:0] data_out);

always @(CLK)
begin
    if(ok)
    begin
        if (!CE_N)
            Clock = CLK;
        else
            Clock = 1'b0;
        end
    end
end

always @(posedge Clock) begin
    latchInputs(~GWE_N, A, DI);
    Gwe_R      <= ~GWE_N;
    Address_R <= A;
    if(!CE_N) begin
        if(A >= 128)
            $display("%m Address:%h is out of bounds",A);
        end
    DataIn_R <= DI;
end
// writing x if address or Gwe_R is not defined... and writing
into memory if its 1
always @(DataIn_R or Address_R or Gwe_R) begin
    measureOutput(memArray_Address_R_);
end
always @(posedge Clock) begin
    executeCommands();
end
endmodule
```

3.2.1.3 Разработка тестов

После создания тестовой системы и написания последовательностей транзакций, необходимо сформировать библиотеку тестов, где каждый тест должен быть унаследован от базового теста — `base_test`. В базовом тесте определяются общие свойства и логика для всех тестов, объявляется окружение и устанавливается конфигурация. Кроме того, здесь также описывается логика вывода статуса завершившегося теста — «Test passed» или «Test failed».

Внутри класса, помимо конструктора и `build_phase`, содержится задача `run_phase`, которая определяет логику запуска базовых и виртуальных последовательностей.

В случае с тестовой системой устройства встроенного самотестирования памяти выбор теста в главную очередь влияет на поведение модели памяти. Для этого в `Makefile` до запуска основного теста, запускается скрипт конфигурации ошибки. Он может выбрать случайную ошибку из списка возможных или запустить сценарий ошибки, указанной пользователем. Результатом работы скрипта является конфигурационный файл для модели памяти, который затем будет использован для инъекции ошибки. В листинге 3.5 приведен скрипт конфигурации ошибки.

Листинг 3.5 – Скрипт конфигурации ошибки

```
#!/usr/bin/env python3
import xml.etree.ElementTree as ET
import random as rn
import sys
import os

rn.seed(int(sys.argv[1]))
row_data=rn.randint(0,int(sys.argv[3]))
column_data=rn.randint(0,int(sys.argv[4]))
module_name=str(sys.argv[5])
print(sys.argv[1])
print(sys.argv[2])
print(module_name)
tree=ET.parse(sys.argv[2])
root=root.getroot()
for role in root.iter('role'):
    role.set('row',str(row_data))
    role.set('column',str(column_data))
for fb in root.iter('faults_binding'):
    fb.set('hier_module_name',module_name)
tree.write(sys.argv[2])
with open('config.mem','w') as mem:

mem.write('{:03x}'.format(row_data)+'\n'+'{:03x}'.format(column_
data))
```

3.2.1.4 Скрипты ошибок

Скрипты инъекции ошибок (Fault Injection Scripts) обычно следуют единой структуре, позволяющей гибко определять условия срабатывания и действия. В общем виде они содержат компоненты:

- `new` – инициализация объекта ошибки (может задавать начальные параметры);
- `parameters` – настраиваемые параметры ошибки (например вероятность появления ошибки);
- `actors` – целевые ошибки, на которые воздействует ошибка;
- `action` – основная логика ошибки: условия срабатывания.

Принцип работы:

- Создается экземпляр ошибки, могут устанавливаться дефолтные параметры;
- Определяются настройки ошибки (например, какие биты заменять, вероятность);
- Указываются, на какие объекты воздействует ошибка (адреса, регистры, ячейки памяти);
- Выполнение действия, при выполнении условия срабатывания.

Пример скрипта для ошибки Transition Fault приведен в листинге 3.6.

Листинг 3.6 – Скрипт ошибки Transition Fault

```
TransitionFault = {  
  new = function()  
    return {};  
  end,  
  
  parameters = {  
    value = "0",  
  },  
  
  actors = {  
    "victim"  
  },  
  
  action = function(state, params, port, now, actors)
```

Продолжение листинга 3.6 – Скрипт ошибки Transition Fault

```
if(string.sub(actors.victim, 1, 1) == params.value) then
    if(not ((string.sub(actors.victim, -1) == 'r') or
(string.sub(actors.victim, -1) == params.value))) then
        return params.value..params.value;
    end
end
return «-«;
end,
}
```

Скрипты ошибок интегрируются в модель памяти при помощи файлов конфигурации. Модель считывает конфигурацию, и в требуемую ячейку подставляет класс ошибки, который в свою очередь является интерпретатором скриптового языка ошибок и обрабатывается данную ячейку в соответствии со скриптом.

3.2.2 Модели информационной базы

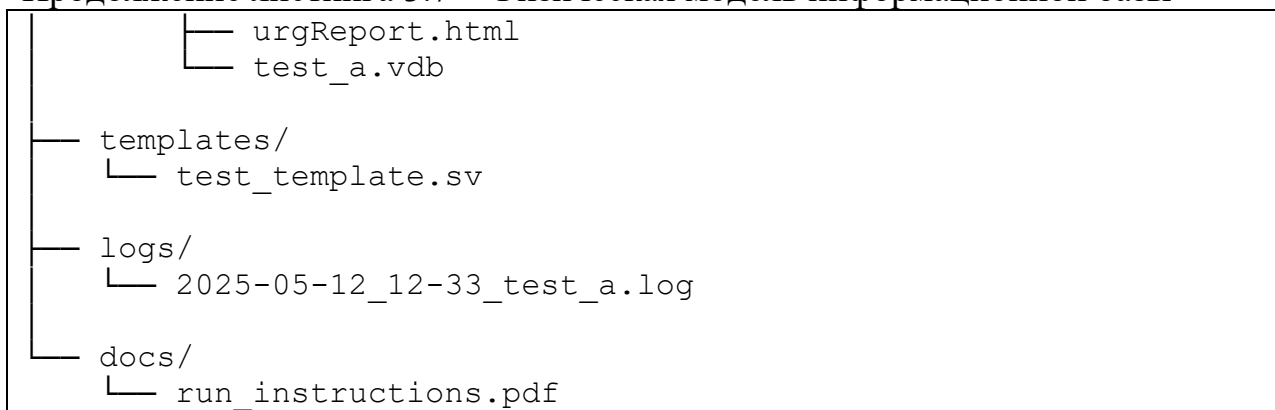
3.2.2.1 Физическая модель информационной базы

Физическая модель показывает, каким образом информация реализуется на уровне файловой системы или хранилища. В данном проекте не используется полноценная СУБД, поскольку данные распределены по файловой структуре, но элементы физической модели всё же структурируются образом, представленном в листинге 3.7.

Листинг 3.7 – Физическая модель информационной базы

```
/project_root/
├── configs/
│   └── test_a.cfg
├── tests/
│   ├── bist_mem_1_p/
│   │   ├── test_a.sv
│   │   └── test_a.log
├── coverage/
│   └── test_a/
```

Продолжение листинга 3.7 – Физическая модель информационной базы



Таким образом, физическая структура отражает логическую модель, но адаптирована под работу с файловыми ресурсами, в том числе на удалённом сервере.

3.2.2.2 Особенности хранения и поиска информации

Поскольку проект рассчитан на работу без централизованной СУБД, особое внимание уделяется:

- стандартизации наименований файлов и каталогов (test_name/date/time);
- использованию индексов и CSV-таблиц для поиска нужных результатов;
- автогенерации файлов метаданных, например, summary.json для хранения параметров теста, статуса и отчёта;
- интеграции с SVN для контроля версий всех объектов информационной базы.

3.2.2.3 Связь модели с программными компонентами

Модели информационной базы напрямую используются следующими компонентами:

- конфигурационными скриптами запуска – читают настройки из файлов cfg и xml;

- симулятором VCS – использует файлы описания тестов и подключений к DUT;
- UVM-тестбенчем – динамически подгружает параметры из модели;
- моделью памяти – в соответствии прочитанными скриптами, выбирается необходимая модель ошибки.

3.2.2.4 Расширяемость модели

Информационная база разработанной системы обладает потенциалом для дальнейшего расширения и адаптации под растущие требования проекта. При необходимости модель может быть дополнена новыми логическими сущностями, отражающими, к примеру, взаимодействие с внешними модулями или детализацию параметров тестов. Это позволит повысить точность описания процессов верификации и улучшить организацию хранения сопутствующих данных.

Текущая структура хранения информации позволяет внедрять формализованные механизмы обмена данными, такие как файловые описания в формате JSON или XML. Это упрощает взаимодействие между компонентами системы и поддерживает консистентность данных при масштабировании тестов и добавлении новых сценариев использования.

Также предусмотрена возможность включения дополнительных форматов представления и хранения данных, а также введения вспомогательных метаданных, обеспечивающих более глубокую классификацию, фильтрацию и последующую интерпретацию результатов тестирования.

Таким образом, заложенная модель информационной базы сохраняет гибкость и пригодность для поэтапного наращивания функциональности без необходимости полной переработки логической или физической структуры.

3.3 Системы классификации и кодирования

В процессе проектирования информационной системы автономной верификации устройства большое значение имеет организация удобного, однозначного и масштабируемого способа хранения, обработки и поиска информации. Для достижения этих целей в проекте реализуется система классификации и кодирования, обеспечивающая структурированность и логическую упорядоченность всех видов данных.

3.3.1 Назначение систем классификации и кодирования

Системы классификации и кодирования в данном проекте применяются для:

- уникальной идентификации всех элементов информационной базы;
- упрощения навигации по структурам хранения данных;
- автоматизации процессов сортировки и фильтрации;
- обеспечения однозначности в идентификации объектов;
- повышения читаемости логов и отчетов;
- поддержки сопоставления элементов системы в различных подсистемах (например, при запуске тестов, анализе покрытий, формировании конфигураций).

3.3.2 Принципы классификации объектов

Классификация данных строится по следующим признакам:

- по типу сущности (например, тест, результат, конфигурация, пользователь, журнал);
- по логической принадлежности (например, регрессионные, функциональные, интерфейсные);
- по уровню абстракции (базовые тесты, композитные тесты, проверки интерфейса);

- по времени выполнения (дата запуска, ревизия конфигурации);
- по целевому устройству или среде выполнения (имя DUT, модель или ревизия).

Эти категории используются для формирования структуры каталогов и наименований файлов, а также для дальнейшего анализа и генерации отчетности.

3.3.3 Принципы кодирования информации

Кодирование элементов выполняется по формализованным шаблонам, которые обеспечивают однозначность и машинную читаемость. Примеры кодов и шаблонов:

- Идентификатор теста: `<устройство>_<имя>_test`, например `bist_mem_1p_saf_test`;
- Конфигурация: `<формат памяти>_<тип ошибки>_cfg`, например `1p_saf_cfg`;
- Результат теста: `res_<дата>_<время>_<код теста>`, например `res_20250512_1130_bist_mem_1p_saf_test`.

В журналах и логах эти коды используются для создания ссылок на другие объекты, упрощая трассировку ошибок и взаимосвязей между сущностями.

3.3.4 Хранение и использование кодов

Формализованные коды и классификационные категории сохраняются в виде:

- имён файлов и каталогов (например, `tests/TSK_FUNC_0032/test.sv`);
- идентификаторов внутри конфигурационных файлов;
- служебных CSV-файлов и JSON-метаописаний;
- тегов в логах, отчётах покрытия и отчётах верификации;

- элементов UI/CLI-интерфейсов при выводе информации пользователю.

3.3.5 Расширяемость системы классификации

Разработанная система классификации и кодирования допускает добавление новых категорий и типов объектов без необходимости пересмотра существующих правил. Это позволяет масштабировать проект в зависимости от появления новых устройств, типов тестирования или требований к автоматизации.

Весь процесс классификации и кодирования документирован, что обеспечивает единообразное использование на всех этапах разработки, тестирования и анализа.

3.4 Обеспечение информационной безопасности

В ходе разработки системы автономной верификации устройства встроенного самотестирования памяти (BIST) были реализованы меры информационной безопасности, направленные на защиту конфиденциальности, целостности и доступности данных. Применённые решения соответствуют современным стандартам и включают организационные, технические и аппаратные методы защиты.

3.4.1 Организационные меры

1. Регламентация доступа и контроль действий:
 - а. внедрена ролевая модель разграничения прав (RBAC), обеспечивающая доступ к проектным данным только уполномоченным сотрудникам.

- б. Использована система контроля версий SVN с ведением журнала изменений и механизмом проверки вносимого кода (Code Review);
 - в. Все участники проекта подписали соглашение о неразглашении (NDA).
2. Политика информационной безопасности:
- а. Разработаны внутренние регламенты, регламентирующие обработку и хранение данных.
 - б. Проведены обучающие семинары по основам кибербезопасности для команды разработчиков.
3. Мониторинг и аудит:
- в. регулярно проводится анализ логов доступа к критическим компонентам системы.
 - г. осуществляется периодический аудит безопасности на соответствие требованиям ГОСТ Р 52069.0-2013 и ISO/IEC 27001.

3.4.2 Технические меры защиты

1. Защита данных:
- а. Все передаваемые данные шифруются с использованием TLS 1.3.
 - б. конфиденциальная информация (алгоритмы тестирования, тестовые векторы) хранится в зашифрованном виде (AES-256).
2. Контроль целостности системы:
- а. Для подтверждения подлинности ПО применяется электронная подпись (ECDSA).
 - б. Проверка целостности прошивки выполняется с помощью хеш-сумм (SHA-3).
3. Защита от несанкционированного доступа и обратной инженерии:

- а. Код, реализованный для ПЛИС, защищён методами обфускации и защищённой загрузки (Secure Boot).
- 4. Безопасная разработка (Secure SDLC):
 - а. На этапе проектирования проведён анализ угроз.
 - б. Реализовано статическое и динамическое тестирование безопасности.
 - в. Проверена устойчивость к атакам по сторонним каналам.

3.4.3 Аппаратные меры защиты

- 1. Физическая безопасность:
 - а. Ограничен доступ к лабораторным стендам и отладочным интерфейсам (JTAG, UART).
 - б. Используются аппаратные модули доверенной загрузки.
- 2. Защита от Fault Injection:
 - а. Реализованы механизмы обнаружения сбоев по питанию и тактированию.

3.4.4 Соответствие стандартам

Разработанная система соответствует следующим стандартам:

- ГОСТ Р 52069.0-2013 (Защита информации).
- ISO/IEC 15408 (Common Criteria) – для сертификации аппаратных решений.
- IEC 62443 (кибербезопасность промышленных систем).

Применение перечисленных мер позволило обеспечить безопасность системы на всех этапах разработки и эксплуатации. Реализованные решения защищают проект от утечек данных и несанкционированного доступа, что подтверждается проведёнными тестами и аудитами.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы была успешно разработана и реализована система автономной верификации устройства встроенного самотестирования памяти, предназначенная для комплексного тестирования и контроля корректности функционирования встроенной подсистемы памяти в составе проектируемых микропроцессоров семейства Эльбрус.

Основной целью проекта являлось создание масштабируемой, автоматизированной и универсальной среды верификации, способной автономно выявлять ошибки в работе BIST, собирать информацию о покрытии и формировать полноценную отчётность. В процессе проектирования был учтён ряд современных требований к верификационным средам, включая поддержку высокоуровневых методологий, возможность имитации различных классов сбоев, автоматическую настройку сценариев и повторяемость результатов.

В рамках работы была разработана полноценная верификационная среда на языке SystemVerilog с использованием методологии UVM, которая обеспечивает модульность, повторное использование компонентов и стандартизованный подход к организации тестов. Применение UVM позволило структурировать верификационную систему, выделить уровни генерации тестов, стимулов, проверки и анализа результатов, а также обеспечить масштабируемость при добавлении новых тестов и функций.

Ключевым техническим достижением проекта стало создание модели памяти с возможностью инъекции ошибок, реализованной на языке C++. Эта модель подключается к основному верификационному окружению через механизм DPI-C, что позволило эффективно использовать низкоуровневую реализацию логики ошибок с сохранением высокого уровня абстракции основной тестовой системы. Разработка этой модели стала одной из наиболее

трудоёмких и значимых частей проекта: был реализован целый набор функций для симуляции различных типов сбоев (временные ошибки, постоянные, с зависимостями от нескольких ячеек и др.), а также механизм конфигурирования сценариев через вспомогательные скрипты на Python.

Дополнительно в рамках проекта были созданы настраиваемые конфигураторы ошибок, которые позволяют пользователю без модификации кода задавать параметры тестирования, управлять типами инъекций и формировать комбинации для различных верификационных условий. Это позволило существенно упростить процесс создания новых тестов и повысить воспроизводимость результатов.

Разработка осуществлялась на локальном рабочем месте разработчика, в то время как все трудоёмкие вычислительные задачи — компиляция, запуск симуляции, сбор покрытия и формирование отчетов — выполнялись на выделенном удалённом сервере, обладающем достаточной производительностью для проведения ресурсоёмких тестов. Такой подход обеспечил высокую эффективность процесса и оптимальное распределение ресурсов.

Для автоматизации и хранения исходных кодов и конфигураций использовалась система контроля версий SVN (Subversion), что позволило отслеживать изменения, восстанавливать предыдущие ревизии и организовать командную работу над проектом в перспективе.

В результате внедрения данной системы были исправлены критические ошибки в работе BIST, ранее не выявленные стандартными средствами тестирования, а также собрано покрытие, отражающее глубину верификации компонентов устройства. Верификационная среда показала высокую степень надёжности и гибкости при проведении серийных тестов.

Важно отметить, что построенная система не является статичной: архитектура тестбенча и структура моделей предусматривают возможность дальнейшего расширения, включая поддержку новых конфигураций памяти,

моделей ошибок, добавление формализованных форматов отчётности и визуализации результатов.

Разработанная система будет внедрена на стадии верификации микропроцессоров семейства Эльбрус и, по предварительным оценкам, позволит значительно снизить трудоёмкость ручного тестирования, повысить точность диагностики ошибок и ускорить цикл выпуска новых аппаратных решений.

Таким образом, поставленные задачи в рамках ВКР были полностью выполнены. Созданная система является практически значимой разработкой, обладающей высоким потенциалом для промышленного применения в области верификации встроенных компонентов цифровых микросхем.

CONCLUSION

During the course of this diploma project, a fully functional autonomous verification system for a built-in self-test memory unit was successfully developed and implemented. This system is intended for comprehensive testing and validation of the embedded memory subsystem in the design of the Elbrus microprocessor family.

The primary objective of the project was to build a scalable, automated, and reusable verification environment capable of autonomously detecting errors in BIST operation, collecting coverage data, and generating complete reports. The system design accounted for modern requirements of verification environments, including support for high-level methodologies, simulation of different fault classes, automatic scenario configuration, and result reproducibility.

As part of the project, a full-featured verification environment was developed using SystemVerilog and the UVM. UVM enabled a modular and reusable structure, a standardized approach to test organization, and easy scaling for new test cases and functionalities. The use of UVM provided clear separation between stimulus generation, checking, and reporting components, streamlining the development process.

One of the key technical achievements was the development of a C++ memory model with built-in error injection capabilities, connected to the SystemVerilog environment via DPI-C. This allowed the project to combine high-performance low-level logic with high-level testbench abstraction. This part of the project was one of the most complex and technically intensive tasks. The model supported simulation of various error types (transient, permanent, multi-bit, etc.), along with a mechanism for configuring test scenarios via dedicated Python scripts.

In addition, error configurators were developed to enable flexible control of fault injection without modifying source code. These tools allowed users to define error parameters, test types, and combinations programmatically, which

significantly simplified the creation of new test scenarios and enhanced repeatability.

While the development was carried out locally on the developer's workstation, all computationally intensive tasks — such as compilation, simulation, coverage collection, and report generation — were executed on a dedicated remote server with high-performance capabilities. This approach ensured resource optimization and increased productivity.

The project used the SVN (Subversion) version control system to manage source code and configuration files, enabling traceability of changes, safe version rollback, and future team collaboration support.

As a result of implementing this verification system, critical issues in BIST behavior were detected and. The verification system demonstrated high reliability and flexibility when executing batch and parameterized test scenarios.

Importantly, the system is designed with extensibility in mind. Its modular testbench architecture and model structure allow for future enhancements, such as support for new memory configurations, additional error types, and the integration of reporting formats and visualization tools.

The developed system is planned to be deployed in the verification flow for Elbrus microprocessors, and is expected to significantly reduce manual testing workload, improve diagnostic accuracy, and shorten the development cycle for new hardware products.

In summary, all the goals set at the beginning of the diploma project were fully achieved. The resulting system is a practically valuable solution, well-suited for industrial application in the verification of embedded components in modern digital microelectronics.

ИСПОЛЬЗОВАННЫЕ ИСТОЧНИКИ

1. Козлов, Н. В. Методы верификации цифровых устройств // Известия вузов. Электроника. 2020. №1. – С. 54–61
2. Clarke, E.M. Model Checking / Clarke, E.M., Grumberg, O., & Peled, D. – MIT Press – 1999
3. ModelSim User's Manual // Mentor Graphics Corporation – 2015
4. Synopsys VCS Functional Verification Solution [Электронный ресурс] – URL: <https://www.synopsys.com> (дата обращения: 23.03.2025)
5. Single-Stop Learning Resource for JasperGold Formal [Электронный ресурс] – URL: cadence.com/cadence_blogs_8/b/cadence-support/posts/jaspergold (дата обращения: 23.03.2025)
6. Questa Intel FPGA Edition Simulation User Guide // Intel – 2023
7. Verilator User's Guide [Электронный ресурс] – URL: <https://verilator.org/guide/latest/> (дата обращения: 23.03.2025)
8. Tessent MemoryBIST Usage Guide and Reference. – USA: Mentor Graphics Corporation – 2012
9. Testing Multi-port Memories: Theory and Practise. – The Netherlands: Delft University of Technology, 2001
10. Functional Memory Faults: A Formal Notation and a Taxonomy. – The Netherlands: Delft University of Technology – 2000
11. ITRS Reports, 2010-2011 [Электронный ресурс] – URL: www.itrs.net (дата обращения: 23.03.2025)
12. Detecting Faults in the Peripheral Circuits and an Evaluation of SRAM Tests. – The Netherlands: Delft University of Technology – 2004
13. Fault Models and Tests for Multi-Port Memories. – The Netherlands: Delft University of Technology – 1997

14. Альфонсо Д. М. Построение алгоритма тестирования и исправления ошибок в блоках кэш-памяти верхнего уровня микропроцессора // Выпускная квалификационная работа. – 2015
- 15.S. Tasiran, K. Keutzer. “Coverage Metrics for Functional Validation of Hardware Designs”. IEEE Design & Test of Computers, 2001.
- 16.Bergeron, J. “Writing Testbenches using SystemVerilog”. Springer, 2006.
- 17.IEEE Std 1800-2023 SystemVerilog Language Reference Manual, 2023.
- 18.Блохина В. Б. Повышение надежности микросистем с использованием BIST-методов / Блохина В. Б., Коноплев Б. Г. – ИЗВЕСТИЯ ТРТУ – 1998 – УДК 621. 3.049.771.14

ПРИЛОЖЕНИЕ

Список приложений:

- Итоговая презентация.

**Федеральное государственное бюджетное образовательное учреждение
высшего образования «Российский технологический университет» МИРЭА**
Институт информационных технологий (ИТ)
Базовая кафедра №250 – МОИС



Выпускная квалификационная работа на тему:

«РАЗРАБОТКА СИСТЕМЫ АВТОНОМНОЙ ВЕРИФИКАЦИИ УСТРОЙСТВА ВСТРОЕННОГО САМОТЕСТИРОВАНИЯ ПАМЯТИ»

Выполнил:

Гордеев Михаил Ильич

Студент группы:

ИВМО-04-23



Руководитель:

Доцент, Уманский Станислав Александрович

2025

ЦЕЛЬ И ЗАДАЧИ РАБОТЫ

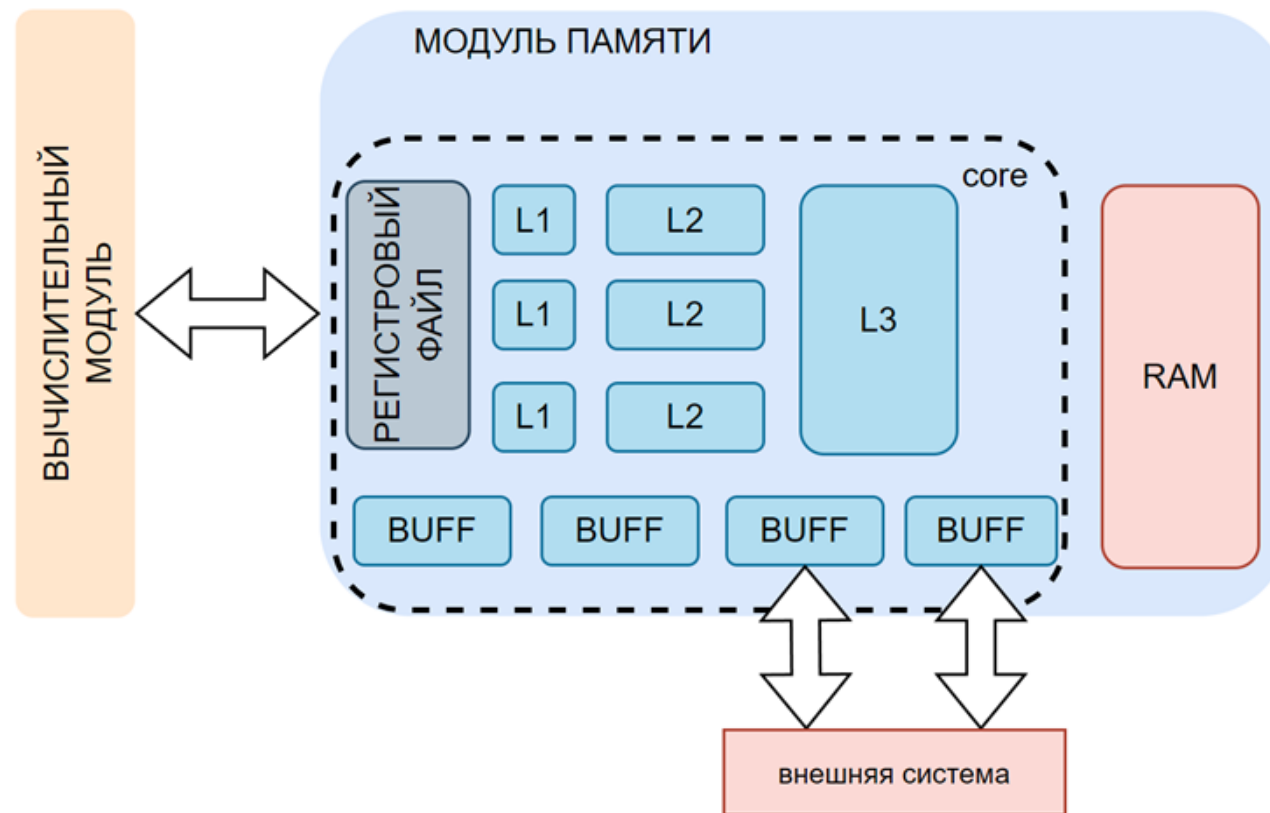
Цель:

- Разработать систему автономной верификации для устройства встроенного самотестирования памяти (Built-In Self-Test, BIST)

Задачи:

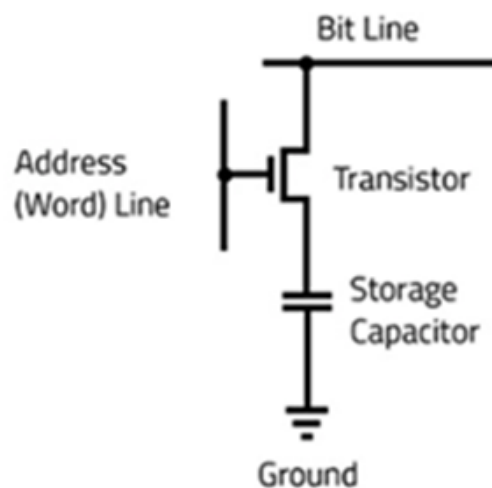
- Анализ проблемы
- Анализ подходов решения
- Реализация системы верификации
- Обзор результатов

МОДЕЛЬ ПАМЯТИ ПРОЦЕССОРА

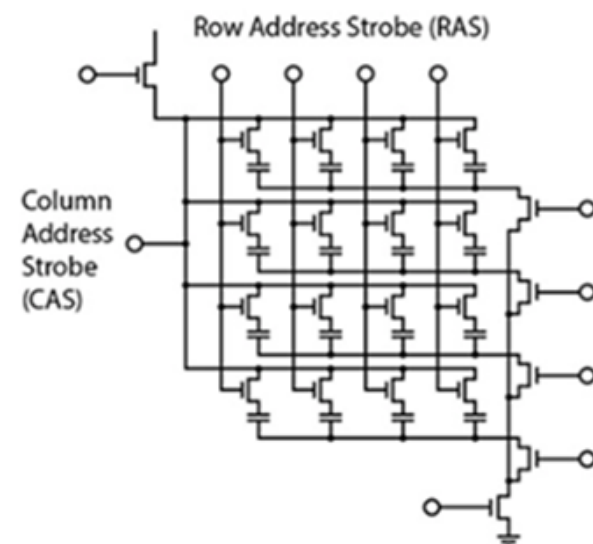


ОШИБКИ ПАМЯТИ

Ошибки одной ячейки	Ошибки нескольких ячеек
SAF – Stuck At Fault	CFst – State Coupling Fault
SOF – Stuck Open Fault	CFds – Disturb Coupling Fault
TF – Transition Fault	CFid – Idempotent Coupling Fault
WDF – Write Destructive Fault	CFin – Inversion Coupling Fault
RDF – Deceptive Read Destructive Fault	CFtr – Transition Coupling Fault
DRF – Data retention Fault	CFwd – Write Destructive Coupling Fault
PF – Parametric Fault	CFdr – Deceptive Read Destructive Coupling Fault



Single Memory Cell

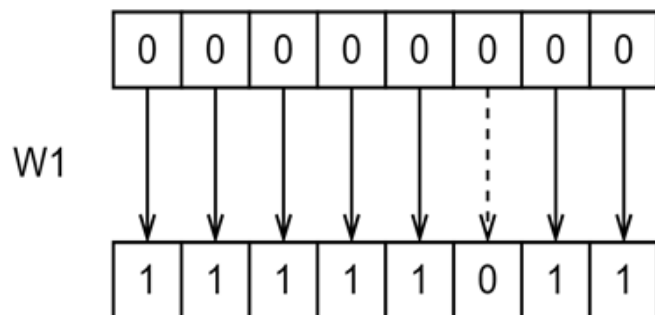


Memory Cell Array

ОШИБКИ ПАМЯТИ

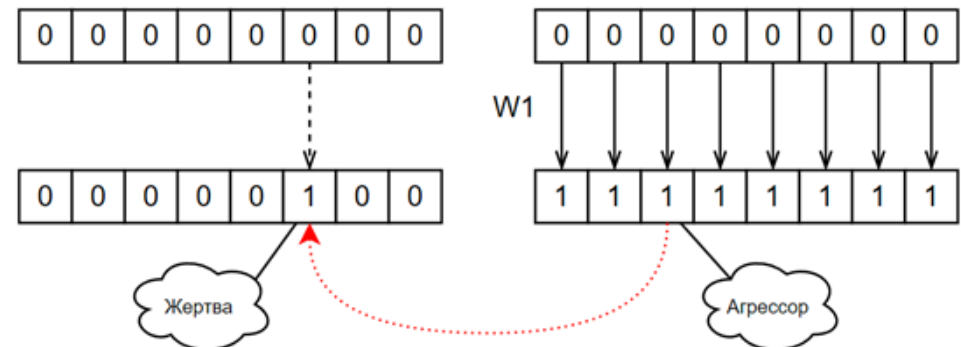
SAF – Stuck At Fault

Ячейка постоянно находится в состоянии логического нуля или логической единицы.

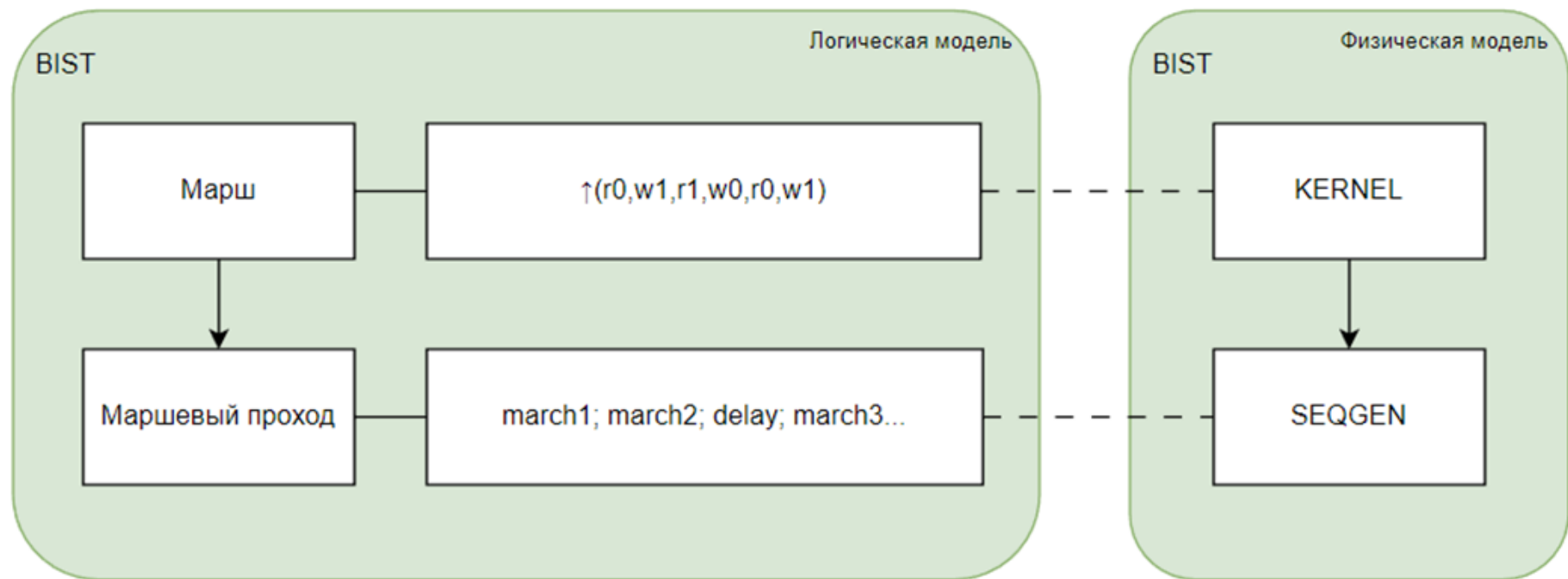


CFst – State Coupling Fault

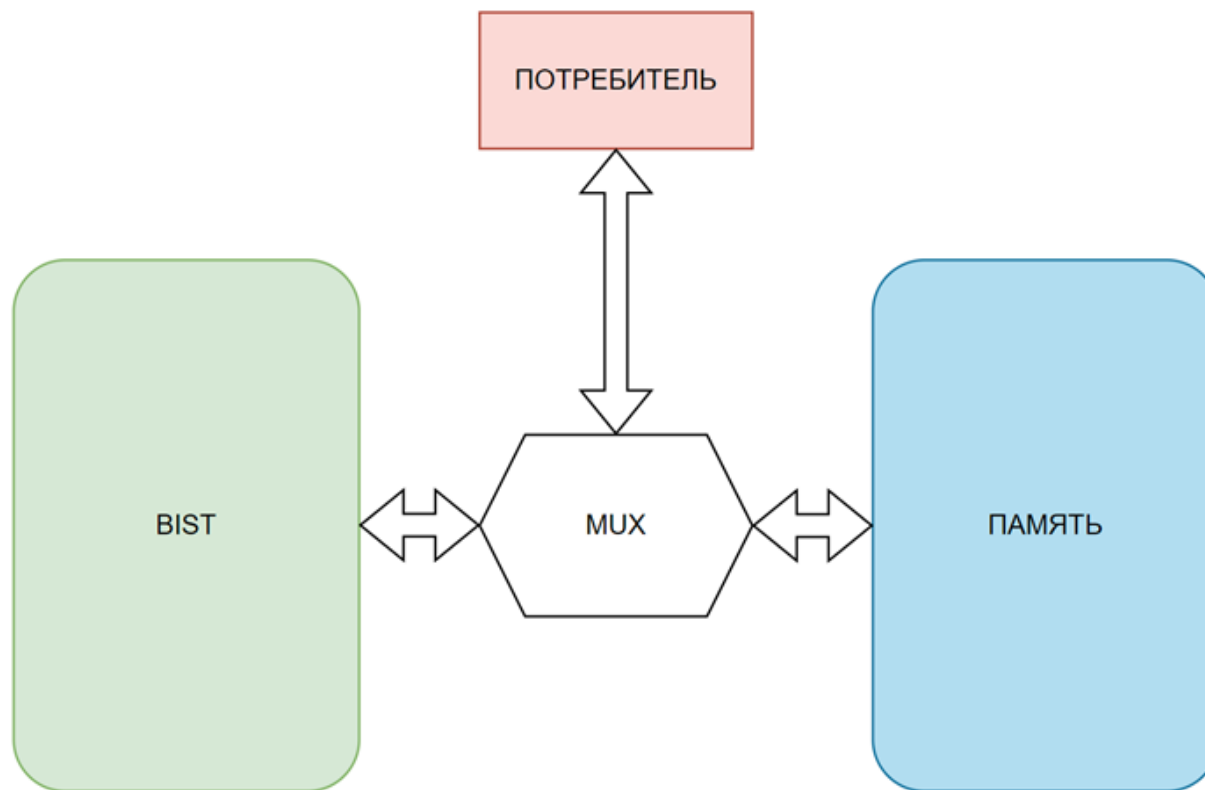
Ячейка переходит в определенное состояние при некотором значении другой ячейки.



BIST



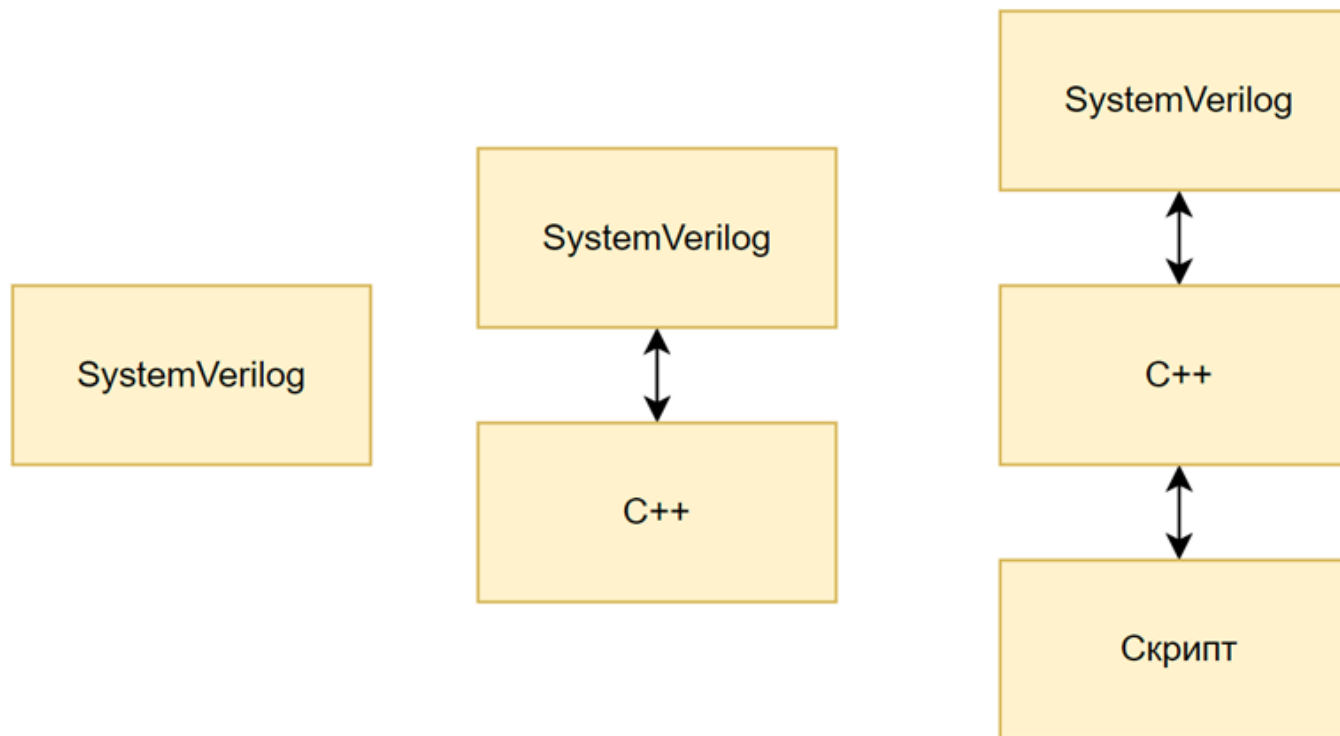
BIST



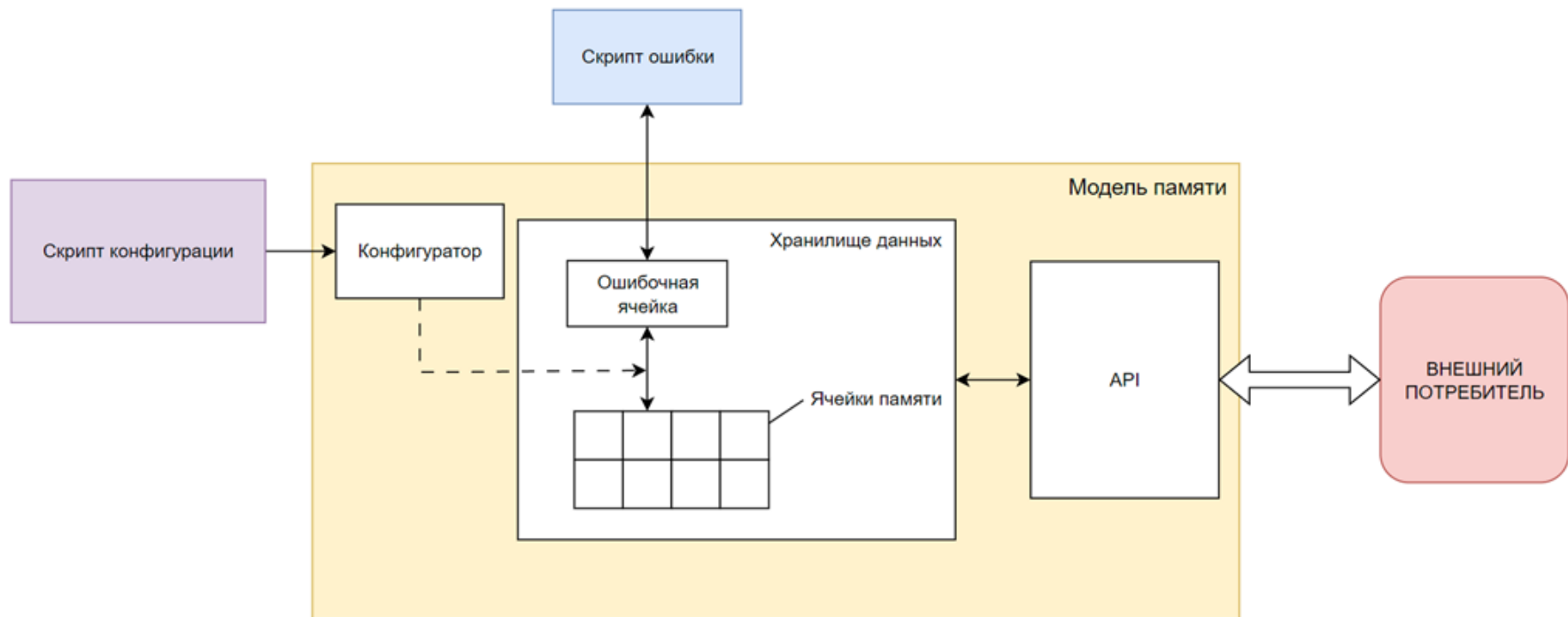
Интерфейсы памяти:

- Шина записи
- Шина данных
- Адрес доступа
- Разрешение на запись
- Сигнал работы
- Прочие интерфейсы

МОДЕЛЬ ПАМЯТИ



МОДЕЛЬ ПАМЯТИ



МОДЕЛЬ ПАМЯТИ

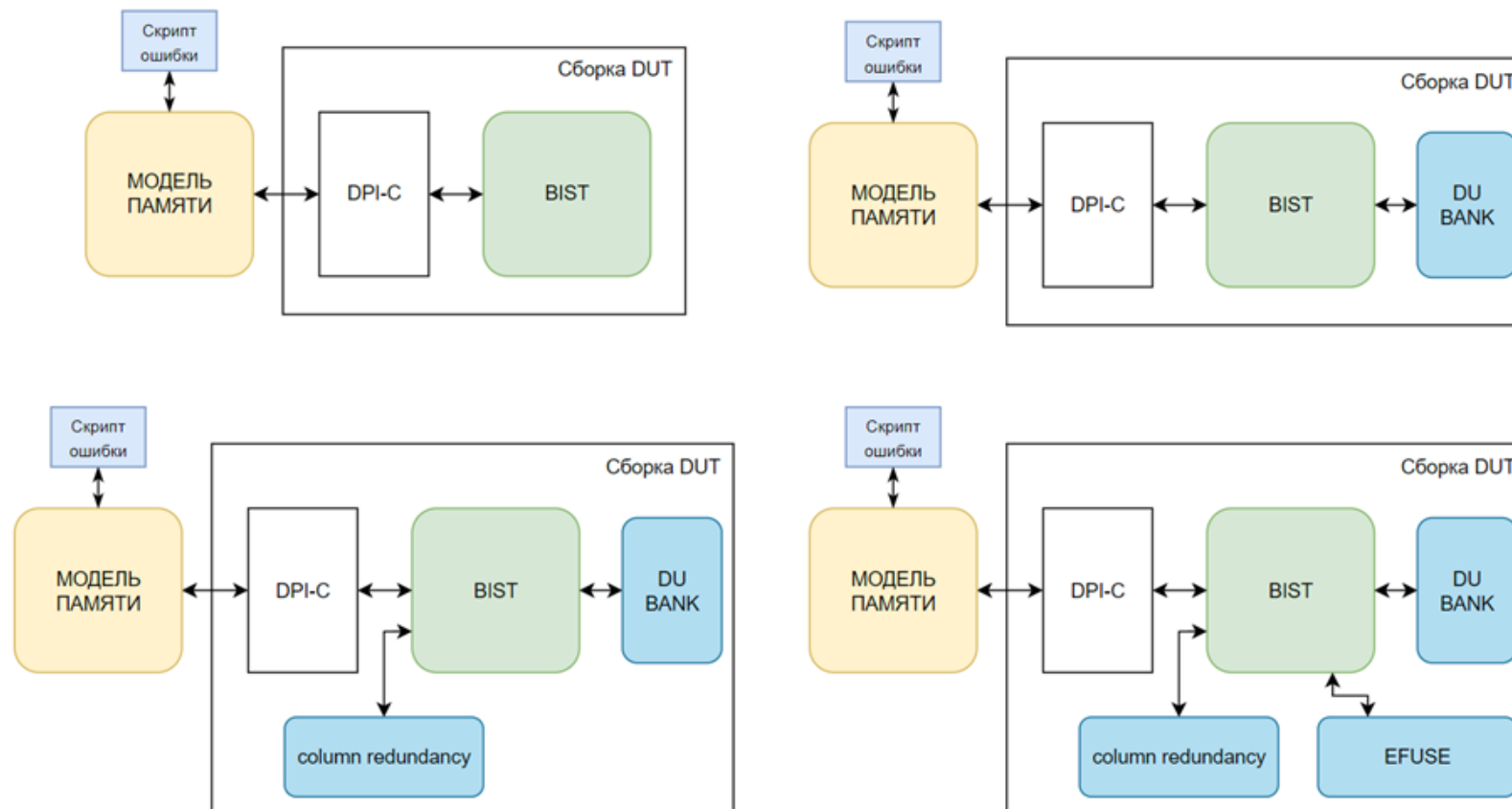
SAF – Stuck At Fault

```
StuckAtFault = {  
  new = function() return {} end,  
  parameters = { value = "0" },  
  actors = { "victim" },  
  action = function(state, params,  
                    port, now, actors)  
    return params.value..params.value  
  end,  
}
```

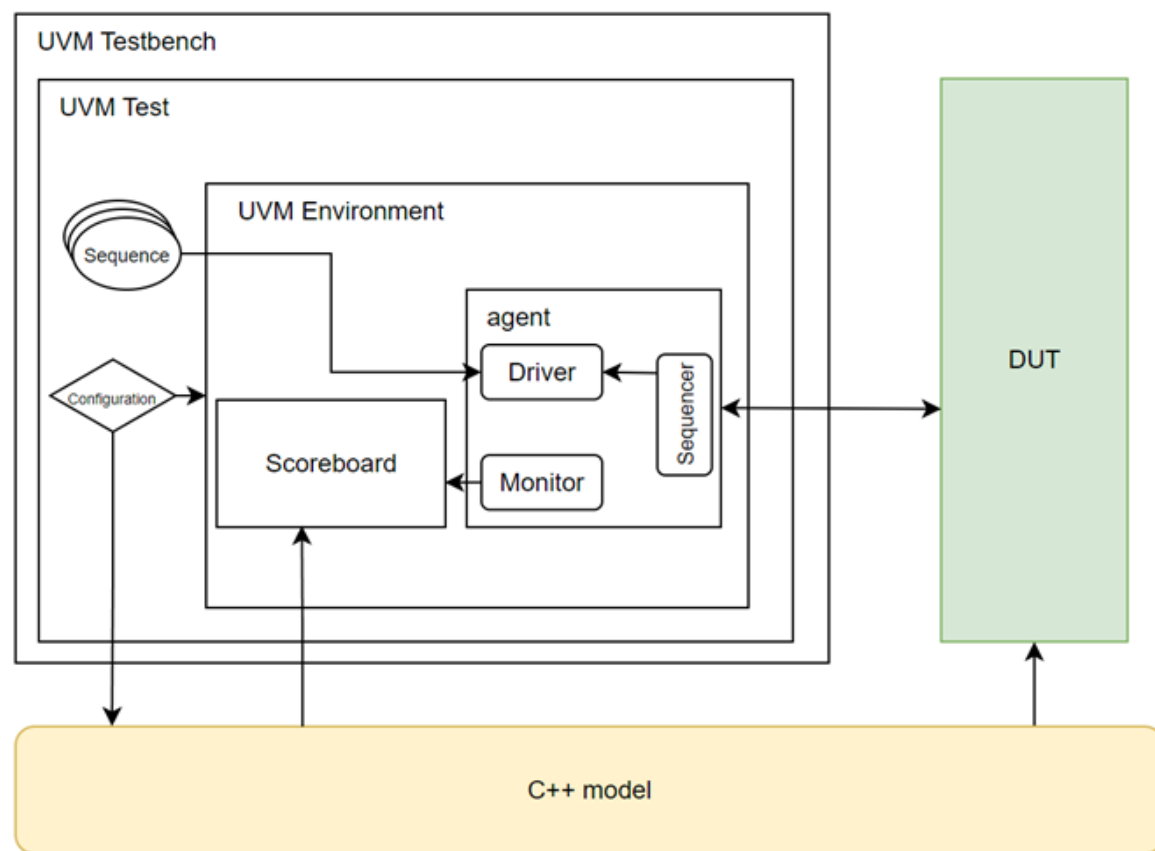
CFst – State Coupling Fault

```
StateCouplingFault = {  
  new = function() return {} end,  
  parameters = { value = "2", react = "2" },  
  actors = { "victim", "aggressor" },  
  action = function(state, params, port, now, actors)  
    return string.sub(actors.aggressor, 1, 1) ==  
    params.react  
    and params.value..params.value  
    or "--"  
  end,  
}
```

ТЕСТОВАЯ СИСТЕМА



ТЕСТОВАЯ СИСТЕМА



РЕЗУЛЬТАТЫ

SYNOPSIS®

Dashboard

[dashboard](#) | [hierarchy](#) | [modlist](#) | [groups](#) | **[tests](#)** | [asserts](#)

Date: Fri Apr 12 16:17:58 2024

User: gordeev_m

Version: O-2018.09-SP1

Command line: urg -dir /mnt/c:/vscode/ghisler/gordeev_m/coverage_reports/0000/MEM1P.vdb/ -elfile

/mnt/c:/vscode/ghisler/gordeev_m/coverage_reports/0000/my_bist.el -elfile

/mnt/c:/vscode/ghisler/gordeev_m/bist_ex.el -report

/mnt/c:/vscode/ghisler/gordeev_m/coverage_reports/0000/mem1p_report

Number of tests: 1

Total Coverage Summary

SCORE	LINE	COND	BRANCH
98.20	100.00	96.97	97.62

Hierarchical coverage data for top-level instances

SCORE	LINE	COND	BRANCH	NAME
98.20	100.00	96.97	97.62	bist_tb_top

Total Module Definition Coverage Summary

SCORE	LINE	COND	BRANCH
89.77	96.08	84.78	88.46

Федеральное государственное бюджетное образовательное учреждение
высшего образования «Российский технологический университет» МИРЭА
Институт информационных технологий (ИТ)
Базовая кафедра №250 – МОИС



Выпускная квалификационная работа на тему:

«РАЗРАБОТКА СИСТЕМЫ АВТОНОМНОЙ ВЕРИФИКАЦИИ УСТРОЙСТВА ВСТРОЕННОГО САМОТЕСТИРОВАНИЯ ПАМЯТИ»

Выполнил:

Гордеев Михаил Ильич

Руководитель выпускной квалификационной работы:
Доцент, **Уманский Станислав Александрович**



2025

**ОТЗЫВ
РУКОВОДИТЕЛЯ ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ**

Тема ВКР: Разработка системы автономной верификации устройства встроенного самотестирования памяти
Автор (студент/ка) Гордеев Михаил Ильич
Институт информационных технологий
Кафедра Математического обеспечения информационных систем
Направление 09.04.01 «Информатика и вычислительная техника»
Магистерская программа «Архитектура вычислительной техники и информационных систем»
Руководитель Уманский Станислав Александрович, доцент кафедры математического обеспечения информационных систем.
(Ф.И.О., место работы, должность, ученое звание, степень)

Оценка соответствия требованиям ФГОС ВО подготовленности автора выпускной работы:

Требования к профессиональной подготовке	Соответствует	В основном соответствует	Не соответствует
осуществляет критический анализ проблемной ситуации и применяет различные виды знаний, научные принципы, методы исследования для решения профессиональных задач	+		
исследует современные проблемы прикладной информатики	+		
управляет проектом на этапах жизненного цикла с учетом командного подхода и обеспечения его эффективности		+	
анализирует и структурирует профессиональную информацию для решения вопросов по проектированию/разработке/модернизации аппаратного и/или программного обеспечения	+		
принимает управленческие решения по результатам проектирования программного обеспечения, структуры базы данных, программных интерфейсов при создании информационных систем		+	
применяет нормативно-технические документы (стандарты и регламенты)	+		

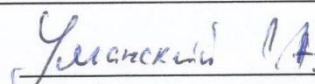
Отмеченные достоинства Работа выполнена на актуальную тему, отражающую современные задачи в области верификации встроенных устройств. Автор уверенно владеет методами функциональной верификации, применяет язык SystemVerilog, библиотеку UVM, а также технологии DPI-C и C++. Построена рабочая система с автоматизацией сборки и тестирования, реализована модель памяти с инъекцией ошибок.

Отмеченные недостатки В работе местами ощущается несистематичность в теоретической части. Отдельные разделы оформления и анализа покрытия могли быть проработаны более последовательно. Тем не менее, недостатки не влияют на завершённость и практическую значимость проекта.

Заключение Считаю, что выпускная квалификационная работа Гордеева Михаила Ильича заслуживает хорошей оценки, а её автор — присвоения квалификации магистр по направлению 09.04.01 «Информатика и вычислительная техника», магистерская программа «Архитектура вычислительной техники и информационных систем».

Руководитель проекта


подпись


фамилия и инициалы

«31» мая 2025 г.

Рецензия
на ВКР на тему
«Разработка системы автономной верификации устройства встроенного
самотестирования памяти»

Гордеева Михаила Ильича

Выпускная квалификационная работа Гордеева М.И. выполнена на актуальную тему, связанную с разработкой системы автономной верификации устройства встроенного самотестирования памяти. В условиях растущей сложности цифровых микросхем и ужесточения требований к их надёжности, особенно на этапе верификационного тестирования, данная тема является важной как с теоретической, так и с прикладной точек зрения.

Автором рассмотрены современные методы и подходы в области функциональной верификации, а также построение среды тестирования с элементами автоматизации и масштабируемости. Особое внимание уделено реализации модели памяти с инъекцией ошибок, что стало наиболее трудоёмкой и значимой частью проекта.

В первой главе представлен всесторонний анализ существующих подходов к верификации встроенных устройств, выделены основные проблемы, обоснована актуальность выбранной архитектуры системы.

Вторая глава посвящена организационно-технологическому обеспечению проекта: охарактеризованы программные и аппаратные средства, процесс разработки, принципы распределённой обработки, а также описаны инструменты контроля версий и удалённой сборки.

В третьей главе подробно описана реализация проектируемой системы. Представлены структура верификационного окружения, взаимодействие компонентов, сценарии использования, форматы документов, модели информационной базы и меры информационной безопасности.

Автор продемонстрировал высокий уровень подготовки в области верификации цифровых устройств, владение современными инструментами разработки, а также способность решать прикладные задачи, связанные с построением масштабируемых и автоматизированных систем тестирования.

Представленная работа может быть использована как база для дальнейших исследований в области верификации, а также внедрена в процессы тестирования микропроцессоров семейства Эльбрус.

Выпускная квалификационная работа Гордеева М.И. соответствует требованиям, предъявляемым к работам данного уровня, может быть рекомендована к защите и заслуживает оценки «хорошо», а её автор — присвоения квалификации магистра по направлению 09.04.01 «Информатика и вычислительная техника».

Рецензент:

профессор магистратуры ФГУП ВНИИ «Центр»
д.э.н, профессор

31.05.25

Балдин К.В.
Заведующий отделом
Управления по работе с персоналом



Балдин К.В.

Справка о проверке на наличие заимствований

Имя файла: 090401_19И1510_ГордеевМИ_ВКР.docx

Автор: Гордеев М.

Заглавие: Разработка системы автономной верификации устройства встроенного самотестирования памяти

Год публикации: 2025

Комментарий: Не указан

Коллекции: Интернет 2.0, Научные статьи 2.0, Википедия, Англоязычная Википедия, Коллекция Энциклопедий, Библиотека Либрусек, Университетская библиотека, Коллекция КФУ, ВКР Российского университета кооперации, Коллекция АПУ ФСИН, Коллекция ПГУТИ, Репозиторий открытого доступа СПб гос. ун-та, ВКР МИРЭА, Научная электронная библиотека "КиберЛенинка", ЦНМБ Сеченова, Авторефераты ВАК, Диссертации ВАК, Диссертации РГБ, Авторефераты РГБ, Готовые рефераты, ФИПС. Изобретения, ФИПС. Полезные модели, ФИПС. Промышленные образцы, Коллекция Руконт, Библиотека им. Ушинского, Готовые рефераты (часть 2), Открытые научные источники, НЭБ, БиблиоРоссика, Правовые документы I, Правовые документы II, Правовые документы III, Собрание законодательства Российской Федерации



Результат проверки

Оценка оригинальности документа: 90%

Оригинальные фрагменты: 89,90%

Обнаруженные заимствования: 7,72%

Цитирование: 2,38%

90%

8%2%

Работу проверил: Фархшатова Г.Р.

Дата: 21.05.25

Подпись:



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий
Кафедра математического обеспечения информационных систем

ЗАКЛЮЧЕНИЕ

ВКР (магистерская диссертация) обучающегося Гордеева Михаил Ильича группы ИВМО-04-23 на тему «Разработка системы автономной верификации устройства встроенного самотестирования памяти» в соответствии с Порядком проведения проверки на объем заимствования и размещения в сети Интернет выпускных квалификационных работ и научных докладов об основных результатах подготовленных диссертаций СМКО МИРЭА 7.5.1/03.П.57–18 прошла автоматизированный анализ в системе РУКОНТЕКСТ Доля авторского текста (оригинальности) в результате автоматизированной проверки составила 90%.

Анализ результата автоматизированной проверки системой РУКОНТЕКСТ и мнение руководителя ВКР о достоверности, фактической доле оригинального текста и степени самостоятельности студента при написании работы позволяют допустить обучающегося Гордеева Михаила Ильича к защите выпускной квалификационной работы.

Руководитель ВКР

 /Уманский С.А. /
«21» 05 2025 г.

Заведующий кафедрой

 /Довгучиц С.И. /
«21» 05 2025 г.

МИРЭА – РОССИЙСКИЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ

Председателю Государственной экзаменационной комиссии ИИТ

«06» июня 2025 г.

СПРАВКА

Студент Гордеев Михаил Ильич № зач.кн. 23И1510

выполнил ВКР на тему: Разработка системы автономной верификации устройства
встроенного самотестирования памяти

и допущен к защите ВКР в Государственной экзаменационной комиссии.

Зав. кафедрой
математического обеспечения
информационных систем

Руководитель ВКР




____ Довгучиц С.И.

____ Уманский С.А.